

## 운영체제 6

남궁명수

|                                                      |    |
|------------------------------------------------------|----|
| [1-1] I/O 장치 .....                                   | 2  |
| [1] 시스템 구조 .....                                     | 3  |
| [2] 표준 장치 .....                                      | 4  |
| [3] 표준 방식 .....                                      | 5  |
| [4] 인터럽트를 이용한 CPU 오버헤드 개선 .....                      | 6  |
| [5] DMA 를 이용한 효율적인 데이터 이동 .....                      | 7  |
| [6] 디바이스와 상호작용하는 방법 .....                            | 10 |
| [7] 운영체제에 연결하기: 디바이스 드라이버 .....                      | 12 |
| [8] 사례 연구: 간단한 IDE 디스크 드라이버.....                     | 14 |
| [9] 역사상의 기록 .....                                    | 17 |
| [10] 요약.....                                         | 18 |
| [1-2] 하드 디스크 드라이브 .....                              | 19 |
| [1] 인터페이스.....                                       | 19 |
| [2] 기본 구조 .....                                      | 21 |
| [3] 간단한 디스크 드라이브.....                                | 24 |
| [4] I/O 시간 계산 .....                                  | 28 |
| [4] 디스크 스케줄링 .....                                   | 31 |
| [5] 요약 .....                                         | 33 |
| [1-3] RAID(Redundant Array of Inexpensive Disk)..... | 38 |
| [1] 인터페이스와 RAID 의 내부 .....                           | 40 |
| [2] 결함 모델 .....                                      | 41 |
| [3] RAID 의 평가 방법 .....                               | 42 |
| [4] RAID 레벨 0: 스트라이핑 .....                           | 43 |
| [5] RAID 레벨 1: 미러링 .....                             | 50 |
| [6] RAID 레벨 4: 패리티를 이용한 공간 절약 .....                  | 55 |
| [7] RAID 레벨 5: 순환 패리티 .....                          | 62 |
| [7] RAID-5 분석 .....                                  | 62 |
| [8] RAID 비교 : 정리.....                                | 63 |
| [9] RAID 와 관련된 다른 흥미로운 주제들 .....                     | 64 |
| [10] 요약.....                                         | 65 |

## [1-1] I/O 장치

핵심 질문: 어떻게 I/O 를 시스템에 통합할까

- 시스템이 I/O 를 어떻게 통합해야 하는가?
- 일반적인 방법은 무엇인가?
- 어떻게 효율저궁로 통합할 수 있을까?

본론(영속성)에 들어가기 전에, 먼저 입력/출력(I/O) 장치의 개념을 소개하고 운영체제가 이러한 장치들과 어떻게 상호작용하는 지 살펴보자.

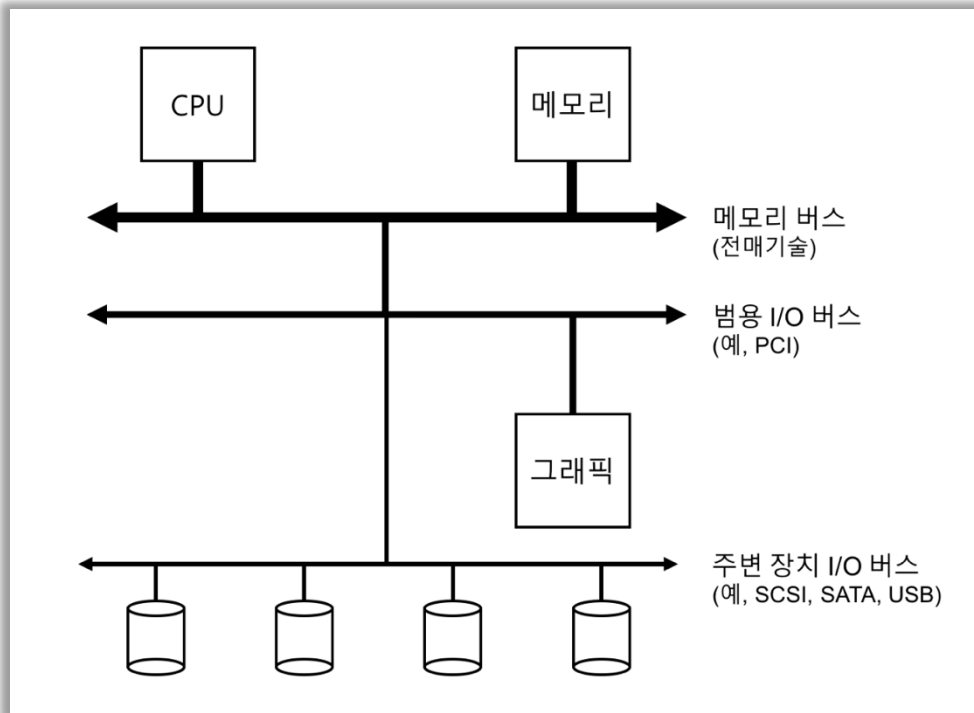
당연한 이야기지만, I/O 는 컴퓨터 시스템에서 매우 중요한 요소이다.

입력이 전혀 없는 프로그램(항상 동일한 결과만 출력하는 프로그램)이나 출력이 없는 프로그램을 생각해보라.

그런 프로그램이 실행되는 것이 과연 어떤 의미가 있을까?

컴퓨터 시스템을 유용하게 사용하려면 입력과 출력이 모두 필요하다는 점은 분명하다.

## [1] 시스템 구조



논의를 시작하기 위해, 그림에 나타난 일반적인 시스템 구조를 살펴보자.

이 구조에서 CPU와 주 메모리가 메모리 버스로 연결되어 있다.

그리고 여러 장치들이랑 범용 I/O 버스에 연결되어 있는데, 현대 시스템에서는 보통 PCI(또는 그 파생 버스)가 사용된다.

그래픽 카드나 고성능 I/O 장치들이 여기에 연결된다.

그 아래에는 SCSI, SATA, USB와 같은 주변장치용 버스가 있다.

이 버스를 통해 디스크나 마우스 같은 상대적으로 느린 장치들이 연결된다.

이러한 계층적 구조가 필요한 이유는 물리적 제약과 비용 때문이다.

- 고속 버스는 짧아야 한다. -> 많은 장치를 연결하기 어렵다.
- 고속 버스는 제작 비용이 비싸다

따라서 시스템 설계자들은 다음과 같은 구조를 선택한다.

- 고성능 장치 -> CPU에 가까이 배치
- 저성능 장치 -> 더 먼 곳에 배치

특히 디스크와 같은 느린 장치를 주변 I/O 버스에 연결하면, 더 많은 장치를 연결할 수 있다는 장점이 있다.

## [2] 표준 장치

이제 가상의 표준 장치를 살펴보고, 이를 효율적으로 사용하기 위해 필요한 요소를 알아보자.

표준 장치는 크게 두 가지 중요한 요소를 가진다.

### (1) 하드웨어 인터페이스

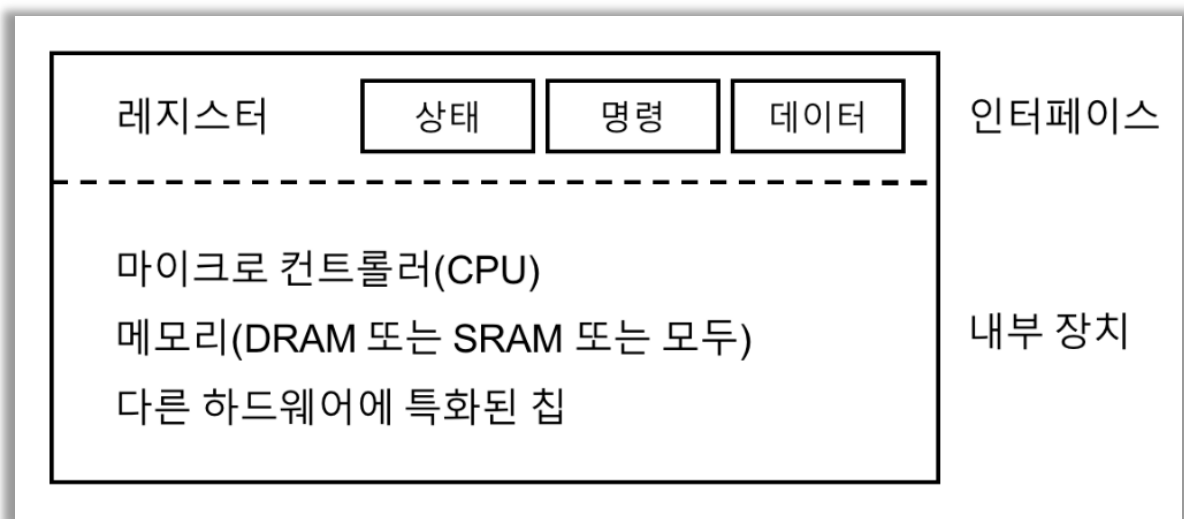
장치는 시스템의 다른 구성 요소와 상호작용하기 위한 인터페이스를 제공해야 한다. 소프트웨어가 인터페이스를 제공하듯, 하드웨어도 명확한 인터페이스를 통해 운영체제가 장치를 제어할 수 있도록 해야 한다.

### (2) 내부 구조

장치는 내부적으로 어떻게 동작할지를 정의하는 구조를 가진다.

- 단순한 장치 -> 몇 개의 하드웨어 칩으로 구성
- 복잡한 장치 -> CPU, 메모리, 특화 칩을 포함

예를 들어, 최신 RAID 컨트롤러는 수십만 줄의 펌웨어(software)가 내부 동작을 정의한다.



### [3] 표준 방식

단순화된 장치 인터페이스는 보통 다음과 같은 세 가지 레지스터로 구성된다.

- 상태 레지스터(Status Register) -> 장치 상태 확인
- 명령 레지스터(Command Register) -> 장치에 명령 전달
- 데이터 레지스터(Data Register) -> 데이터 송수신

운영체제는 이 레지스터들을 읽고 쓰는 방식으로 장치를 제어한다.

#### 장치와 운영체제의 상호작용 과정

장치가 작업을 수행할 때, 일반적으로 다음과 같은 절차를 따른다.

1. 폴링(polling)
  - 상태 레지스터를 반복적으로 읽어 장치가 준비되었는지 확인
2. 데이터 전달
  - 운영체제가 데이터 레지스터에 데이터를 씀
  - 이때 CPU가 직접 관여하면 -> Programmed I/O
3. 명령 전달
  - 명령 레지스터에 작업 명령 기록
4. 완료 대기
  - 다시 폴링을 수행하며 작업 완료 여부 확인

#### 이 방식의 문제점

이 방식은 간단하지만 비효율적이다.

가장 큰 문제는 폴링이다.

- CPU가 계속 루프를 돌며 상태를 확인함
- 다른 작업을 하지 못하고 시간을 낭비함
- 특히 I/O 장치는 매우 느리기 때문에 비효율이 심함

#### 핵심 질문: 폴링의 비용을 어떻게 줄일 수 있을까?

- 어떻게 하면 자주 폴링하지 않으면서도 운영체제가 장치 상태를 효율적으로 확인하고 CPU 오버헤드를 줄일 수 있을까?

#### [4] 인터럽트를 이용한 CPU 오버헤드 개선

이미 오래전부터 엔지니어들은 장치와의 상호작용을 개선하기 위해 인터럽트(interrupt)라는 개념을 개발하였다.

인터럽트 방식에서는 장치를 계속 폴링하는 대신, 운영체제가 출력 작업을 요청한 프로세스를 블록 상태로 전환시키고 CPU 를 다른 프로세스에게 양도한다.

이후 장치가 작업을 완료하면 하드웨어 인터럽트를 발생시키고, CPU 는 운영체제가 미리 정의해둔 인터럽트 서비스 루틴(interrupt Service Routine, ISR), 즉 인터럽트 핸들러를 실행한다.

이 인터럽트 핸들러는 운영체제 코드의 일부로서 다음과 같은 역할을 수행한다.

- I/O 작업 완료 처리(예: 데이터 읽기, 에러 코드 확인)
- I/O 를 기다리던 프로세스를 깨움
- 해당 프로세스가 다시 실행될 수 있도록 준비

#### CPU 사용률을 높이는 핵심: 연산과 I/O 의 중첩

인터럽트를 사용하면 CPU 연산과 I/O 작업을 동시에 겹쳐서 수행할 수 있다.

#### [인터럽트 없는 경우(폴링 방식)]

- 프로세스 1 이 실행됨
- 디스크 I/O 요청 발생
- CPU 는 계속 장치 상태를 폴링(polling)하며 대기
- I/O 완료 후 다시 프로세스 1 실행
- CPU 가 아무 일도 하지 않고 기다림(비효율)

#### [인터럽트 사용하는 경우]

- 프로세스 1 실행 -> I/O 요청
- 운영체제는 프로세스 1 을 블록
- 프로세스 2 를 CPU 에서 실행
- 디스크 작업 완료 -> 인터럽트 발생
- 프로세스 1 을 깨워 다시 실행
- CPU 와 디스크가 동시에 활용됨 -> 효율 증가

인터럽트가 항상 좋은 것은 아니다.

인터럽트가 항상 최적의 해결책은 아니다.

예를 들어:

- 장치가 매우 빠른 경우(한 번의 폴링으로 끝남)
  - 인터럽트를 사용하면 오히려 느려짐
  - 이유:
    - 문맥 교환(context switch) 비용
    - 인터럽트 처리 비용

이 경우는 폴링이 더 효율적

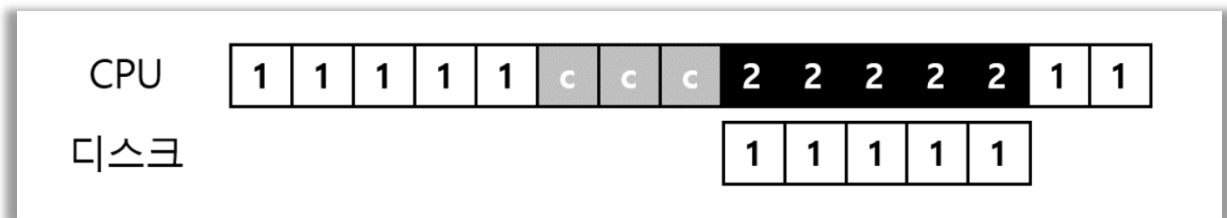
#### [5] DMA 를 이용한 효율적인 데이터 이동

앞에서 설명한 표준 방식에는 한 가지 더 고려해야 할 문제가 있다.

많은 양의 데이터를 디스크로 전달할 때 Programmed I/O(PIO)를 사용하면, CPU 가 단순한 데이터 복사 작업에 많은 시간을 소비하게 된다.

이로 인해 CPU 는 원래 처리해야 할 다른 작업(프로세스 실행 등)에 사용할 수 있는 시간을 낭비하게 된다.

#### PIO 방식의 문제



시간 흐름을 보면 다음과 같은 상황이 발생한다.

- 프로세스 1 실행 중
- 디스크에 데이터를 쓰기 위해 I/O 요청 발생
- CPU 가 메모리 -> 디스크로 데이터를 한 워드씩 직접 복사
- 그림에서는 이 과정을 C로 표시

복사가 끝난 후에야 디스크가 실제로 I/O 작업을 시작하고,

그제서야 CPU 는 다른 작업을 수행할 수 있게 된다.

즉, CPU 가 데이터 복사 작업에 묶여 있음(비효율)

**핵심 질문: PIO 의 오버헤드를 어떻게 줄일 것인가?**

- PIO 방식에서는 CPU 가 데이터 이동에 너무 많은 시간을 사용한다.
- 이 작업을 줄이고 CPU 를 더 효율적으로 활용하려면 어떻게 해야 할까?

**해결책: DMA(Direct Memory Access)**

이 문제를 해결하기 위한 방법이 바로 DMA(직접 메모리 접근)이다.

DMA 는 시스템 내부에 존재하는 특수 하드웨어(DMA 엔진)로, CPU 의 개입 없이 메모리와 장치 간 데이터 전송을 담당한다.

### DMA 동작 방식

DMA 는 다음과 같은 순서로 동작한다.

#### 1. 운영체제가 DMA 설정

- 메모리 상 데이터 위치
- 전송할 데이터 크기
- 대상 장치
- 이 정보들을 DMA 엔진 전달

#### 2. CPU 는 다른 작업 수행

- 설정이 끝나면 CPU 는 더 이상 관여하지 않음
- 다른 프로세스를 실행 가능

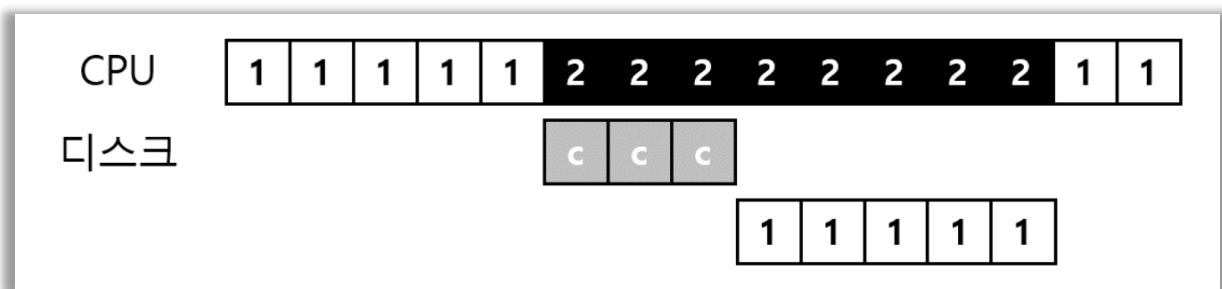
#### 3. DMA 가 데이터 전송 수행

- 메모리 <-> 장치 간 데이터 복사 담당

#### 4. 전송 완료 후 인터럽트 발생

- DMA 컨트롤러가 인터럽트를 발생
- 운영체제 “작업 완료” 알림

### DMA 사용 시 변화



시간 흐름을 보면:

- 데이터 복사는 DMA 컨트롤러가 처리
- CPU 는 그 동안 자유롭게 다른 작업 수행



예를 들어:

- 프로세스 1 -> I/O 요청
- DMA 가 데이터 복사 진행
- CPU -> 프로세스 2 실행
- DMA 완료 -> 인터럽트 발생
- 다시 프로세스 1 실행

CPU 활용도가 크게 증가

## [6] 디바이스와 상호작용하는 방법

지금까지 I/O 요청을 처리할 때 효율성의 중요성에 대해 살펴보았다.

하지만 효율성 외에도 고려해야 할 중요한 문제가 하나 더 있다.

- 바로 운영체제와 장치가 실제로 어떻게 정보를 주고받는가이다.

핵심 질문: 장치와 운영체제는 어떻게 통신하는가?

- 하드웨어와 장치는 어떤 방식으로 데이터를 주고받을까?
- 특별한 명령어가 존재하는가, 아니면 다른 방법이 있을까?

### 장치와 통신하는 두 가지 방법

지금까지 사용되어 온 대표적인 방법은 두 가지이다.

#### (1) I/O 명령어 방식(Port-Mapped I/O)

가장 오래된 방식으로, 과거 IBM 메인프레임에서도 사용되던 방법이다.

- 운영체제가 특수한 I/O 명령어를 사용하여 장치와 통신
- 장치의 레지스터에 직접 데이터를 쓰거나 읽음

예를 들어 x86에서는:

- in 명령어 -> 장치에서 데이터 읽기
- out 명령어 -> 장치에 데이터 쓰기

동작 방식:

- 데이터가 들어 있는 레지스터 지정
- 장치를 식별하는 포트(port) 지정
- 명령어 실행 -> 장치와 통신 수행

#### 중요한 특징: 특권 명령어

이러한 I/O 명령어는 대부분 특권 명령어(privileged instruction)이다.

즉, 운영체제만 실행 가능

- 사용자 프로그램이 디스크를 직접 제어할 수 있다면
- 시스템 전체가 위험해짐(보안 문제, 시스템 붕괴 등)

따라서 장치 제어는 반드시 하드웨어만 담당

## (2) 메모리 매핑 I/O (Memory-Mapped I/O)

두 번째 방법은 장치 레지스터를 메모리처럼 다루는 방식이다.

- 장치 레지스터를 메모리 주소 공간에 매핑
- 일반적인 메모리 접근처럼 사용 가능

즉,

- load(읽기) -> 장치 레지스터 읽기
- store(쓰기) -> 장치 레지스터 쓰기

하지만 실제로는:

- 메모리가 아니라 장치와 연결됨
- 하드웨어가 해당 주소를 감지하여 장치로 전달

### 두 방식 비교

| 구분    | I/O 명령어 방식       | 메모리 매핑 I/O              |
|-------|------------------|-------------------------|
| 접근 방식 | 전용 명령어 (in, out) | 일반 메모리 접근 (load, store) |
| 구현    | 별도 명령어 필요        | 기존 명령어 재사용              |
| 사용성   | 다소 불편            | 더 직관적                   |
| 사용 여부 | 현재도 사용           | 현재도 사용                  |

## [7] 운영체제에 연결하기: 디바이스 드라이버

이제 마지막으로 다룰 문제는,  
서로 다른 인터페이스를 가진 다양한 장치들과 운영체제를 어떻게 연결할 것인가이다.

예를 들어, 파일 시스템을 생각해보자.

- SCSI 디스크
- IDE 디스크
- USB 이동식 드라이브

이런 다양한 장치 위에서 동일한 파일 시스템이 동작해야 한다.  
하지만 파일 시스템이 각 장치의 구체적인 명령 방식에 종속되도록 만들고 싶지는 않다.

**핵심 질문: 어떻게 장치에 독립적인(장치 중립적인) 운영체제를 만들 수 있을까?**

- 장치마다 다른 구현을 숨기고
- 운영체제는 공통된 방식으로 장치를 다루게 할 수 있을까?

**해결 방법: 추상화(Abstraction)**

이 문제를 추상화를 통해 해결할 수 있다.  
운영체제의 가장 아래 계층에는 장치의 동작 방식을 실제로 이해하는 소프트웨어가 존재한다.  
이것이 바로 “디바이스 드라이버(Device Driver)”이다.

**디바이스 드라이버의 역할**

- 장치와의 구체적인 상호작용을 담당
- 복잡한 동작을 내부에 캡슐화
- 운영체제 나머지 부분에는 단순한 인터페이스 제공

즉, 장치의 복잡함을 숨겨주는 번역기 역할을 수행한다.

## Linux 구조 예시(파일 시스템 스택)

Linux 의 구조를 보면 다음과 같은 흐름이다.

1. 응용 프로그램/파일 시스템
  - 디스크 종류를 전혀 모름
2. 범용 블록 계층(Generic Block Layer)
  - read/write 요청을 받아 처리
3. 디바이스 드라이버
  - 요청을 해당 장치에 맞게 변환
4. 실제 장치
  - 명령 수행

파일 시스템은 “읽어라/써라”만 요청하고,  
어떻게 하는지는 드라이버가 알아서 처리한다.

### 추상화의 단점

이러한 캡슐화는 장점만 있는 것은 아니다.

문제: 장치가 가진 고급 기능을 활용하기 어려움

예를 들어,

- SCSI 장치 -> 매우 상세한 에러 정보 제공
- IDE/ATA 장치 -> 단순한 에러만 제공

하지만 운영체제는 공통 인터페이스만 사용하기 때문에  
결국 파일 시스템은 단순한 에러 코드(EIO)만 받게 됨  
SCSI 의 풍부한 정보는 활용되지 못함

### 현실적인 문제: 드라이버 코드의 비중

흥미로운 사실:

- 운영체제 코드의 70% 이상이 디바이스 드라이버
- Linux, Windows 모두 동일한 경향

즉, “운영체제 = 드라이버 덩어리”라고 봐도 될 정도

### 또 다른 문제: 안정성

많은 드라이버는

- 커널 전문가가 아닌 사람들이 작성
- 품질이 일정하지 않음

결과:

- 버그 발생
- 커널 크래시의 주요 원인

## [8] 사례 연구: 간단한 IDE 디스크 드라이버

좀 더 깊이 이해하기 위해 실제 장치인 IDE 디스크 드라이버를 살펴보자.  
여기서는 단순화된 구조와 함께, xv6 운영체제의 예시를 기반으로 설명한다.

### IDE 디스크 인터페이스 구조

IDE 디스크는 다음과 같은 4 개의 주요 레지스터를 제공한다.

- Control
- Command Block
- Status
- Error

이 레지스터들은 x86 의 in, out 명령어를 통해  
특정 I/O 포트 주소(예: 0x1F0, 0x1F7)에 접근하여 읽고 쓸 수 있다.

### 주요 레지스터 구성

#### Command Block Registers

- 0x1F0 -> 데이터 포트
- 0x1F1 -> 에러 정보
- 0x1F2 -> 섹터 개수
- 0x1F3 ~ 0x1F5 -> LBA 주소 (디스크 위치)
- 0x1F6 -> 드라이브 선택(마스터/슬레이브)
- 0x1F7 -> 명령/상태

#### Status 레지스터(0x1F7)

디스크 상태를 나타냄

- BUSY -> 작업 중
- READY -> 준비됨
- DRQ -> 데이터 요청
- ERROR -> 오류 발생

#### Error 레지스터(0x1F1)

ERROR 발생 시 원인 확인:

- BBK -> 불량 블록
- UNC -> 복구 불가능한 오류
- ABRT -> 명령 중단
- 기타 다양한 오류 상태

## IDE 디스크 동작 과정

장치와 상호작용하는 기본 흐름은 다음과 같다.

### (1) 장치 준비 대기

- Status 레지스터를 반복해서 읽음
- BUSY 가 아니고 READY 상태가 될 때까지 대기

폴링 사용

### (2) 명령 인자 설정

- 섹터 개수
- LBA 주소(읽거나 쓸 위치)
- 드라이브 선택(마스터/슬레이브)

이 값을 Command Block 레지스터에 기록

### (3) I/O 시작

- Command 레지스터(0x1F7)에 READ 또는 WRITE 명령 기록

디스크 작업 시작

### (4) 데이터 전송(쓰기 기준)

- DRQ 상태가 될 때까지 대기
- 데이터 포트(0x1F0)에 데이터 기록

### (5) 인터럽트 처리

- 각 섹터마다 인터럽트 발생 가능
- 또는 모든 작업 후 한 번만 인터럽트 발생(최적화)

### (6) 에러 처리

- Status 확인 -> ERROR 비트 체크
- ERROR 레지스터 읽어 상세 원인 확인

## xv6 IDE 드라이버 구조

xv6에서는 IDE 드라이버가 다음과 같은 함수들로 구성된다.

### ide\_rw()

- 읽기/쓰기 요청 처리
- 요청 큐에 넣거나 바로 실행
- 요청 완료까지 프로세스는 sleep 상태

### ide\_start\_request()

- 실제 디스크에 명령 전달
- in, out 명령어 레지스터 접근

### ide\_wait\_ready()

- 디스크가 준비될 때까지 대기
- (Status 레지스터 확인)

### ide\_intr()

- 인터럽트 발생 시 실행
- 데이터 읽기 수행(읽기 요청인 경우)
- 대기 중인 프로세스 깨움
- 다음 요청 처리 시작



## [9] 역사상의 기록

이 장을 마치기 전에, 지금까지 다룬 I/O 관련 핵심 개념들의 역사적 배경을 간단히 살펴보자.

### 인터럽트의 역사

인터럽트는 매우 오래된 개념으로, 초기 컴퓨터에서도 이미 사용되었다.

- 1950년대 UNIVAC에서도 인터럽트 벡터 방식이 존재
- 다만 정확한 최초 도입 시점은 명확하지 않음

초기 컴퓨터 역사 기록이 불안전해서 “누가 먼저 만들었는지”는 확실하지 않다.

### DMA의 기원

DMA의 최초 도입에 대해서는 의견이 갈린다.

- 일부 -> DYSEAC
- 일부 -> IBM SAGE

하지만 분명한 사실은 1950년대 중반에는 이미

- 메모리 <-> 장치 직접 통신
- 작업 완료 시 인터럽트 발생

이런 구조가 존재했다는 것

### 왜 역사 추적이 어려운가?

- 초기 컴퓨터는 널리 알려지지 않은 실험적 기계가 많았음
- 문서화 부족
- 여러 곳에서 동시에 비슷한 아이디어 등장

예:

- TX-2 -> 벡터 기반 인터럽트 최초 도입 주장(확실하지 않음)

### 핵심 통찰

이 개념들은 사실 그렇게 “혁명적인 발명”이라기 보다는 자연스럽게 등장한 해결책

- CPU는 빠름
- I/O 장치는 느림

따라서:

- 기다리는 동안 CPU는 다른 일을 해야함
- -> 인터럽트
- -> DMA

즉, 성능 차이를 해결하려다 나온 필연적인 구조

## [10] 요약

### I/O 효율성 문제

- I/O 느림 -> CPU 낭비 발생
- 이를 해결하기 위한 기술 필요

### 핵심 기술

- 인터럽트
  - 폴링 대신 사용
  - CPU 와 I/O 를 동시에 활용 가능
- DMA
  - 데이터 복사를 CPU 대신 수행
  - CPU 개입 최소화

### 장치 접근 방식

- I/O 명령어(in, out)
- 메모리 매핑 I/O

둘 다 장치 레지스터 접근 방식

### 디바이스 드라이버

- 장치별 차이를 숨김
- OS 를 장치 독립적으로 만들
- 핵심 역할: 추상화 + 캡슐화

## [1-2] 하드 디스크 드라이브

이전 장에서는 I/O 장치 전반과 운영체제가 I/O 장치가 어떻게 상호작용하는지 살펴보았다.

이제부터는 특정 장치인 하드 디스크 드라이브(HDD)를 좀 더 자세히 알아본다.

하드 디스크 드라이브는 오랫동안 컴퓨터 시스템에서 영구적인 데이터 저장소 역할을 해왔다.

또한 파일 시스템 기술은 대부분 디스크의 동작 방식을 기반으로 설계되었기 때문에, 디스크의 내부 동작을 이해하는 것은 매우 중요하다.

**핵심질문:** 디스크는 데이터를 어떻게 저장하고, 어떻게 접근하는가?

- 데이터는 어떤 구조로 저장되는가?
- 인터페이스는 어떻게 생겼는가?
- 실제 접근 방식은 어떻게 이루어지는가?
- 디스크 스케줄링은 어떻게 성능을 개선하는가?

### [1] 인터페이스

현대 하드 디스크의 인터페이스는 생각보다 단순하다.

#### 기본 구조: 섹터 기반 저장

디스크는 다음과 같이 구성된다.

- 섹터(sector) 단위로 데이터 저장
- 각 섹터 크기: 512 바이트

디스크 전체는 이렇게 볼 수 있다.

- 0 번 섹터 ~  $n-1$  번 섹터
- 즉, 연속된 배열 구조

결론:

- 디스크 = 거대한 배열(array)

#### 멀티 섹터 작업

- 한 번에 여러 섹터 읽기/쓰기 가능
- 예: 파일 시스템은 보통 4KB 단위로 처리

### 중요한 개념: 원자성(Atomicity)

디스크 제조사가 보장하는 것은 512 바이트 쓰기만 원자적  
즉,

- 성공 -> 전체 다 씀
- 실패 -> 하나도 안 씀

### 문제: 찢긴 쓰기(Torn Write)

큰 데이터를 쓰는 도중:

- 갑자기 전원 꺼짐 -> 일부만 저장됨
- 이를 찢긴 쓰기라고 한다.

### 디스크의 암묵적 규칙(계약 불문율)

디스크 인터페이스에 명시되지는 않지만,  
사용자들이 일반적으로 가정하는 중요한 특성이 있다.

#### (1) 가까운 데이터 접근이 더 빠름

- 인접한 블록 -> 빠름
- 멀리 떨어진 블록 -> 느림

#### (2) 순차 접근이 가장 빠름

- 연속된 블록 읽기/쓰기 -> 매우 빠름
- 랜덤 접근 -> 훨씬 느림

이 차이는 매우 큼(성능 설계 핵심 포인트)

## [2] 기본 구조

현대 하드 디스크의 내부 구조를 이해해 보자.

### (1) 플래터(Platter)

디스크의 가장 핵심 구성 요소는 플래터이다.

- 원형의 딱딱한 판
- 표면에 자기적 성질을 이용해 데이터 저장

#### 특징

- 하나 이상의 플래터로 구성
- 각 플래터는 앞 뒤 2 개의 표면(surface)을 가짐
- 재질: 알루미늄 같은 단단한 물질
- 표면에는 얇은 자석층이 입혀져 있음

전원이 꺼져도 데이터가 유지되는 이유

### (2) 스피들(Spindle) & 회전

플래터는 스피들(회전축)에 고정되어 있다.

- 모터와 연결되어 계속 회전
- 속도 단위: RPM(분당 회전 수)

일반적인 속도:

- 7,200RPM ~ 15,000 RPM

중요한 건 “속도”보다 한 바퀴 도는 데 걸리는 시간이 중요하다.

- 10,000RPM → 약 6ms(밀리초)

### (3) 트랙(Track) & 섹터(Sector)

데이터는 플래터 표면에 다음과 같이 저장된다.

트랙(Track)

- 플래터 위의 동심원 형태의선
- 하나의 원 = 하나의 트랙

섹터(Sector)

- 트랙을 나눈 작은 단위
- 실제 데이터 저장 단위(보통 512B)

구조

플래터 표면

- └─── 트랙 (원형)
- └─── 섹터 (조각)

특징

- 트랙은 매우 촘촘함
- 수백 개 트랙 = 머리카락 두께 수준

### (4) 디스크 헤드(Disk Head)

데이터를 읽고 쓰기 위해서는 디스크 헤드가 필요하다.

역할:

- 읽기 -> 자기 패턴 감지
- 쓰기 -> 자기 상태 변경

중요한 특징

- 각 표면마다 헤드 1 개가 존재

### (5) 디스크 암(Disk Arm)

디스크 헤드는 디스크 암에 연결되어 있다.

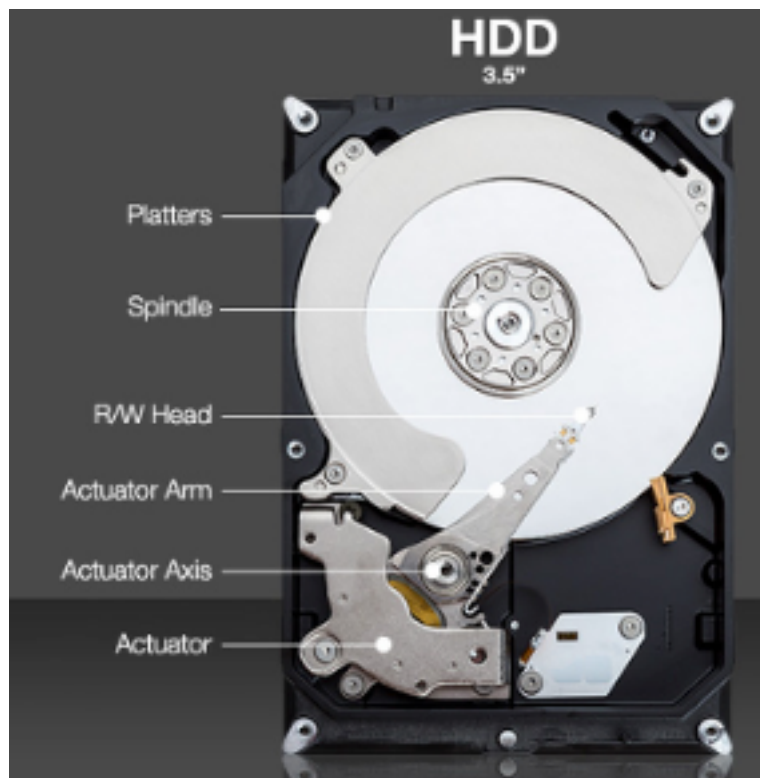
역할: 헤드를 움직여 원하는 트랙으로 이동

즉, 특정 위치 데이터 접근 = 헤드를 해당 트랙으로 이동시키는 것

## 전체 구조 이해

### [플래터 (회전)]

- |—— 트랙 (원형 경로)
- |     └—— 섹터 (데이터 조각)
- └—— 헤드 (읽고 쓰기)
- ↑
- 디스크 암 (이동)



### [3] 간단한 디스크 드라이브

디스크 동작을 이해하기 위해 아주 단순한 모델부터 생각해보자.

#### (1) 단일 트랙 디스크(기초 모델)

- 트랙 1 개
- 섹터 12 개(0~11)
- 각 섹터: 512B

디스크는 회전하고,  
헤드는 특정 위치에 고정되어 있음

#### (2) 회전 지연(Rotational Delay)

예: 헤드가 섹터 6 위에 있고 -> 섹터 0 을 읽으려는 경우  
디스크는 해야 할 일:

- 그냥 돌면서 기다림

#### 핵심 개념: 회전 지연

- 원하는 섹터가 헤드 아래로 올 때까지 기다리는 시간
  - 한 바퀴 시간 =  $R$
  - 평균 지연 =  $R/2$

예:

- 10,000 RPM -> 6ms
- 평균 대기 -> 3ms

#### 최악의 경우

- 거의 한 바퀴 기다림



### (3) 멀티 트랙: 탐색 시간(Seek Time)

현실에서는 트랙이 하나가 아니라 수백만 개 존재한다.

탐색이란?

- 헤드를 원하는 트랙으로 이동시키는 과정

탐색 과정 (4 단계)

1. 가속
2. 등속 이동
3. 감속
4. 정밀 위치 조정(setting)
  - 특히 중요한 것
    - 정확히 위치해야 함
    - 안정화 시간: 약 0.5~2ms

핵심: 탐색 + 회전 = 디스크가 느린 시간

### (4) 실제 요청 처리 흐름

예: 섹터 11 읽기

1. 탐색(seek)
  - 해당 트랙으로 이동
2. 회전 지연(rotation delay)
  - 섹터가 올 때까지 대기
3. 전송(transfer)
  - 데이터 읽기/쓰기

디스크 접근 시간 공식

- 전체 시간:

Total Time = Seek Time + Rotation Delay + Transfer Time

이 3 개가 디스크 성능의 핵심

## (5) 추가 최적화 개념들

### 트랙 스큐(Track Skew)

문제:

- 트랙 이동 후 -> 다음 섹터가 이미 지나가 있음

해결:

- 일부러 섹터 위치를 약간 밀어 배치

결과:

- 바로 이어서 읽기 가능
- 불필요한 회전 지연 감소

### 멀티 존(Multi-Zone Recording)

- 바깥 트랙 -> 더 큼 -> 더 많은 섹터
- 안쪽 트랙 -> 더 적은 섹터

이유: 원 둘레 차이 때문

결과: 바깥쪽이 더 빠름

### 디스크 캐시(Track Buffer)

- 크기: 약 8~16MB
- 최근 데이터 저장

특징: 한 섹터 읽을 때 -> 트랙 전체 읽어서 캐싱

효과: 같은 트랙 접근 -> 매우 빠름

## (6) 쓰기 캐싱 방식

### Write-Through

- 디스크에 실제 기록 후 완료

안전하지만 느림

### Write-back

- 메모리에 쓰면 바로 완료 처리

빠르지만 위험

- 전원이 거지면 데이터 손실 가능

### (7) 차원 분석(Dimensional Analysis)

- 단위로 계산하는 방법

#### 예 1: 회전 시간

10,000 RPM → 한 바퀴 시간:

$$= 60,000 \text{ ms} / 10,000$$

$$= 6 \text{ ms}$$

#### 예 2: 전송 시간

100MB/s 에서 512KB 전송 ( $\text{시간} = \frac{\text{데이터 크기}}{\text{전송 속도}}$ )

$$= \text{약 } 5 \text{ ms}$$

|         |                 |
|---------|-----------------|
| 1 초     | 1,000 밀리초(ms)   |
| 1 밀리초   | 1,000 마이크로초(us) |
| 1 마이크로초 | 1,000 나노초(ns)   |

| 지수       | 값           | 단위 | 의미            |
|----------|-------------|----|---------------|
| $2^8$    | 256         | B  | 1 Byte 경우의 수  |
| $2^{10}$ | 1,024       | B  | 1 KB          |
| $2^{12}$ | 4,096       | B  | 4 KB (페이지 크기) |
| $2^{16}$ | 65,536      | B  | 64 KB         |
| $2^{20}$ | 1,048,576   | B  | 1 MB          |
| $2^{30}$ | 약 $10^9$    | B  | 1 GB          |
| $2^{40}$ | 약 $10^{12}$ | B  | 1 TB          |

#### [4] I/O 시간 계산

이제 디스크의 성능을 수식으로 분석할 수 있다.

##### 핵심 공식

디스크 I/O 시간은 다음 3 가지 요소로 구성된다.

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

의미:

- $T_{seek}$  -> 헤드 이동 시간(탐색)
- $T_{rotation}$  -> 원하는 섹터 기다리는 시간
- $T_{transfer}$  -> 실제 데이터 전송 시간

##### I/O 속도 공식

$$R_{I/O} = \frac{Size_{Transfer}}{T_{I/O}}$$

즉, 전송량 + 시간 = 성능

##### 실험 시나리오

두 가지 워크로드 비교:

###### (1) 랜덤 접근(Random I/O)

- 4KB 랜덤 위치 읽기
- DB 같은 시스템에서 흔함

###### (2) 순차 접근(Sequential I/O)

- 연속된 데이터 읽기
- 파일 읽기/영상 스트리밍 등

##### 디스크 비교

| 항목    | Cheetah (고성능) | Barracuda (용량형) |
|-------|---------------|-----------------|
| RPM   | 15,000        | 7,200           |
| 평균 탐색 | 4ms           | 9ms             |
| 전송 속도 | 125MB/s       | 105MB/s         |

### (1) 랜덤 I/O 계산

Cheetah

- 탐색: 4ms
- 회전: 2ms
- 전송: 0.03ms

총 시간: 약 6ms  
성능: 약 0.66 MB/s

Barracuda

총 시간: 약 13.2ms  
성능: 약 0.31MB/s

### (2) 순차 I/O 계산

조건:

- 100MB 연속 읽기
- 탐색 + 회전은 한 번만 발생

결과

- Cheetah -> 약 125MB/s
- Barracuda -> 약 105 MB/s

거의 디스크 최대 속도

| 유형 | Cheetah   | Barracuda |
|----|-----------|-----------|
| 랜덤 | 0.66 MB/s | 0.31 MB/s |
| 순차 | 125 MB/s  | 105 MB/s  |

충격적인 사실(랜덤 vs 순차)

- Cheetah -> 약 200 배 차이
- Barracuda -> 약 300 배 차이

## 핵심 인사이트

### (1) 디스크 성능의 본질

병목은 전송이 아니라:

- 탐색(seek)
- 회전(rotation)

### (2) 순차 접근이 압도적으로 빠른 이유

- 탐색 1 번
- 회전 1 번
- 이후 쪽 읽기

비용 거의 없음

### (3) 랜덤 접근이 느린 이유

- 매번: 탐색, 회전

비용 반복 발생

## 디자인 교훈(중요)

“디스크는 반드시 순차적으로 사용하라”

- 랜덤 I/O → 성능 폭망
- 큰 블록으로 묶어서 처리
- 데이터는 연속 배치

## 추가 인사이트

디스크 시장 2 가지

### 1. 고성능(cheeta)

- 빠름
- 비쌈

### 2. 대용량(Barracuda)

- 느림
- 저렴

용도에 따라 선택

#### [4] 디스크 스케줄링

##### 핵심 개념

디스크 스케줄러는 I/O 요청의 처리 순서를 결정하며, 탐색 시간과 회전 지연을 예측할 수 있기 때문에 SJF(짧은 작업 우선)원칙을 적용할 수 있다.

##### 스케줄링 알고리즘 비교

###### (1) SSTF(최단 탐색 시간 우선)

- 현재 헤드 위치에서 가장 가까운 트랙의 요청을 먼저 처리
- 운영체제는 실제 트랙 구조를 모르므로, 실용적으로는 NBF(가장 가까운 블럭 우선) 방식으로 대체
- 문제점: 헤드 근처 요청이 계속 들어오면 멀리 있는 요청이 영구히 처리되지 않는 기아 현상 발생

###### (2) SCAN/엘리베이터 알고리즘

- 헤드가 한 방향으로 쪽 이동하며 처리 -> 끝에 도달하면 반대 방향으로 이동(스위프)
- 기아 현상 해결
- 변종들:
  - F-SCAN: 스위프 중에는 큐를 동결, 새 요청은 다음 스위프에 처리 -> 기아 현상 완화
  - C-SCAN: 항상 한 방향(바깥 -> 안)으로만 스위프, 반대 방향은 이동만 함
- 한계: 회전 지연을 고려하지 않아 SJF에 완전히 부합하지 않음

###### (3) SPTF(최단 위치 잡기 우선)

- 탐색 시간 + 회전 지연을 모두 고려해 가장 빠르게 도달할 수 있는 요청을 우선 처리
- 현대 드라이브에서 탐색과 회전 시간이 비슷한 수준이므로 효과적
- 운영체제는 헤드의 정확한 회전 위치를 알 수 없어 드라이브 내부 컨트롤러에서 구현

### 기타 중요 개념

- I/O 병합: 인접한 블록 요청을 하나로 묶어 디스크 요청 수를 줄임
- 작업 보전 방식: 요청이 있으면 즉시 디스크로 전송
- 작업 비보전 방식: 잠시 기다려 더 효율적인 요청이 오면 함께 처리(예측 스케줄링)

### 현대 시스템의 역할 분담

- 운영체제 -> 여러 요청을 묶어 디스크로 전달
- 디스크 컨트롤러 -> SPTF 방식으로 내부에서 최적 순서 결정



## [5] 요약

### (1) CPU <-> I/O 장치 효율 문제

- I/O 장치는 느림
- CPU 가 계속 기다리면서 낭비 발생

### (2) 해결 방법 1: 인터럽트

#### 핵심 아이디어

- CPU 가 기다리지 말고 다른 작업 수행
- I/O 끝나면 장치가 CPU 에게 알림

#### 흐름

1. 프로세스 -> I/O 요청
2. OS -> 프로세스 block
3. CPU -> 다른 프로세스 실행
4. I/O 완료 -> 인터럽트 발생
5. ISR 실행 -> 프로세스 깨움

#### 장점

- CPU 활용률 증가(I/O 와 병렬처리)

#### 단점

- 인터럽트 + 컨텍스트 스위칭 비용 큼
- 너무 빠른 장치에는 오히려 손해

#### 결론

- 느린 장치 -> 인터럽트
- 빠른 장치 -> 폴링

### (3) 해결 방법 2: DMA

문제(PIO)

- CPU 가 직접 데이터 복사 -> 낭비

DMA 해결

- CPU 대신 DMA 가 데이터 복사

흐름

1. OS -> DMA 설정(주소, 크기 등)
2. DMA -> 메모리 <-> 디스크 직접 전송
3. 완료 -> 인터럽트 발생

효과

- CPU 완전 해방
- 진짜 병렬 처리 가능

데이터 이동은 DMA, CPU 는 계산

### (4) 장치와 통신 방법

I/O 명령어 방식

- in, out
- 레지스터 직접 접근
- 특권 명령어(OS 만 가능)

Memory-mapped I/O

- 장치를 메모리처럼 사용
- load/store 로 접근

둘 다 현재 사용됨

## (5) 디바이스 드라이버

문제: 장치마다 인터페이스 다름

응용 프로그램



파일 시스템



Generic Block Layer



디바이스 드라이버



하드웨어

핵심:

- OS 는 장치 몰라도 됨
- 드라이버가 다 처리

현실

- OS 코드의 70%가 드라이버
- 버그의 주범

## (6) 디스크 구조

구성 요소

- Platter: 데이터 저장
- Trak: 동심원
- Sector: 최소 단위(보통 512B)
- Head: 읽기/쓰기
- Arm: 헤드 이동
- Spindle: 회전

### (7) 디스크 성능의 핵심 요소

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

의미:

| 요소       | 설명        |
|----------|-----------|
| Seek     | 트랙 이동 시간  |
| Rotation | 회전 대기     |
| Transfer | 실제 데이터 읽기 |

대부분 시간은 seek + rotation

### (8) 랜덤 vs 순차 I/O

랜덤 I/O

- 매번 seek + rotation 발생
- 엄청 느림

순차 I/O

- 한 번만 seek
- 엄청 빠름

실제 차이

- 최대 200~300 배

결론: 디스크는 무조건 순차 접근

### (9) 디스크 스케줄링

목표: I/O 시간 최소화

SSTF(Shortest Seek Time First)

- 가장 가까운 요청 먼저
- 장점: 빠름
- 단점: 기아

SCAN(엘리베이터 알고리즘)

- 한 방향으로 쪽 이동하며 처리
- 장점: 공정성 증가
- 엘리베이터처럼 움직임

C-SCAN

- 한 방향만 사용(순환)

SPTF

- Seek + Rotation 둘 다 고려
- 현실적으로 최고지만
- OS에서는 구현 어려움(디스크 내부에서 수행)

(10) 추가 핵심 개념

I/O 병합(Merging)

- 연속 요청 -> 하나로 합침

성능이 증가한다.

Anticipatory Scheduling

- 일부러 기다림
- 더 좋은 요청 받기 위해

항상 바로 실행이 최선은 아니다.

### [1-3] RAID(Redundant Array of Inexpensive Disk)

핵심 질문: 대량이면서 빠르고 신뢰할 수 있는 디스크를 어떻게 만들 수 있을까?

- 어떻게 하면 대용량이면서 고속이고 신뢰할 수 있는 저장 시스템을 만들 수 있을까?
- 핵심 기술은 무엇인가?
- 그리고 서로 다른 접근법들 사이의 절충안은 무엇인가?

디스크를 사용할 때 우리는 종종 더 빠르기를 바란다.

I/O 작업은 느리기 때문에 전체 시스템의 병목이 되기도 한다.

또한 우리는 디스크의 용량이 더 크기를 바란다.

점점 더 많은 데이터가 온라인에서 접근 가능하도록 저장되면서, 우리의 디스크는 점점 더 가득 차게 된다.

그리고 우리는 디스크가 더 안정적으로 동작하기를 바란다.

디스크가 고장날 경우, 백업이 없다면 소중한 데이터를 잃어버릴 수 있다.

이번 장에서는 RAID(Redundant Array of Inexpensive Disk)라고 더 잘 알려진 기술을 소개한다.

이 기술은 여러 개의 디스크를 조합하여 고속이면서 대용량이고 신뢰할 수 있는 디스크 시스템을 만든다.

RAID 는 1980 년대 말, UC 버클리에서 처음 소개되었다.(David Patterson Randy Katz, 그리고 당시 학생이던 Grath Gibson 이 주도하였다.)

그 시기에는 더 나은 저장 시스템을 만들기 위해 여러 개 디스크를 활용하는 다양한 아이디어들이 여러 연구진에 의해 제안되었다.

겉보기에는 RAID 는 하나의 디스크처럼 보인다.

즉, 읽고 쓸 수 있는 블록들의 집합으로 보인다.

하지만 내부를 살펴보면 RAID 는 여러 개의 디스크와 메모리(휘발성 및 비휘발성), 그리고 이를 관리하기 위한 하나 이상의 프로세서로 구성된 복잡한 시스템이다.

즉, RAID 의 하드웨어는 컴퓨터 시스템과 매우 유사하며, 디스크 그룹을 관리하기 위한 전용 시스템이라고 볼 수 있다.

RAID 는 단일 디스크에 비해 여러 가지 장점을 제공한다.

첫 번째 장점은 성능이다.

- 여러 디스크를 병렬로 사용하여 I/O 시간이 크게 개선된다.

두 번째 장점은 용량이다.

- 데이터가 증가함에 따라 더 많은 디스크 공간을 활용할 수 있다.

세 번째 장점은 신뢰성이다.

- 데이터를 여러 디스크에 분산 저장함으로써, 하나의 디스크가 고장나더라도 데이터를 보호할 수 있다.

이때 중복성(redundancy) 기술을 사용하면, 하나의 디스크가 고장나더라도 시스템은 문제없이 계속 동작할 수 있다.

RAID 의 또 다른 중요한 특징은, 이러한 장점들은 시스템에 투명하게 제공한다는 것이다.

호스트 시스템은 RAID 를 단지 하나의 큰 디스크로 인식한다.

이 투명성의 의미는, 단 한줄의 소프트웨어 변경 없이도 기존 디스크를 RAID 로 교체할 수 있다는 것이다.

운영체제와 응용 프로그램은 그대로 동작한다.

이처럼 투명성은 RAID 의 보급성을 크게 향상시켰으며, 사용자와 관리자가 호환성 문제를 걱정하지 않고 RAID 를 사용할 수 있게 한다.

이제 RAID 의 중요한 요소들을 살펴보자.

먼저 인터페이스와 장애 모델을 설명하고,

그 다음으로 용량, 신뢰성, 성능이라는 세 가지 핵심 기준을 중심으로 RAID 를 평가하는 방법을 다룬다.

마지막으로 RAID 의 설계와 구현과 관련된 중요한 추가 이슈들을 살펴볼 것이다.

**팁: 투명성은 시스템 도입을 쉽게 만든다.**

시스템에 새로운 기능을 추가할 때는, 그 기능이 투명하게 적용될 수 있는지 항상 고려해야 한다.

즉, 기존 시스템의 다른 부분을 수정하지 않고도 추가할 수 있어야 한다.

만약 기존 소프트웨어를 전부 다시 작성해야 하거나 하드웨어 사양을 크게 바꿔야 한다면, 그 기술은 널리 사용되기 어렵다.

RAID 는 이러한 점에서 매우 좋은 사례이다.

이 투명성 덕분에 RAID 는 성공적으로 널리 보급될 수 있었다.

예를 들어, 관리자는 기존의 SCSI 디스크 대신 RAID 장치를 사용할 수 있으며, 호스트 컴퓨터나 운영체제는 전혀 변경할 필요가 없다.

이처럼 설치(배포)의 용이성을 확보한 것이 RAID 성공의 중요한 이유이다.

## [1] 인터페이스와 RAID 의 내부

상위 계층에 존재하는 파일 시스템에게 RAID 는 크고(희망적으로) 빠르며, (또한 희망적으로) 신뢰할 수 있는 디스크처럼 보인다.

단일 디스크가 그러하듯이 RAID 역시 선형적인 블록들의 배열로 보이며, 파일 시스템(또는 다른 클라이언트)은 각 블록을 읽거나 쓸 수 있다.

파일 시스템이 RAID 에 논리적 I/O 를 요청하면, RAID 는 내부적으로 어떤 디스크(또는 여러 디스크)에 접근해야 요청을 완료할 수 있는지 계산한 뒤, 하나 또는 그 이상의 물리적 I/O 를 발생시킨다.

이 물리적 I/O 의 정확한 특성은 RAID 레벨에 따라 달라지며, 이에 대해서는 이후에 다룰 것이다.

간단한 예로, 각 블록에 대해 서로 다른 디스크에 두 개의 복사본을 유지하는 RAID 를 생각해보자.

이와 같은 미러링 기반 RAID 시스템에서 블록에 쓰기를 수행할 경우, 하나의 논리적 I/O 에 대해 두 개의 물리적 I/O 를 실행해야 한다.



RAID 시스템은 보통 별도의 하드웨어 장치(박스) 형태로 존재하며, 호스트 시스템과는 SCSI 나 SATA 와 같은 표준 인터페이스를 통해 연결된다.

내부적으로 RAID 는 상당히 복잡하다.

RAID 의 동작을 제어하는 펌웨어를 실행하는 마이크로컨트롤러가 있으며, 블록을 읽고 쓰는 과정에서 버퍼로 사용되는 DRAM 같은 휘발성 메모리가 포함된다.

또한 경우에 따라서는 쓰기를 안전하게 버퍼링하기 위한 비휘발성 메모리를 포함하기도 하며, 일부 RAID 레벨에서 필요한 패리티 계산을 수행하기 위한 전용 논리 회로를 갖추고 있기도 하다.

상위 계층의 관점에서 보면 RAID 는 하나의 특수한 컴퓨터 시스템이라고 볼 수 있다.

프로세서와 메모리, 그리고 디스크를 갖고 있으며, 일반적인 응용 프로그램을 실행하는 대신 RAID 동작을 위한 전용 소프트웨어를 실행한다는 점만 다르다.

## [2] 결함 모델

RAID 를 이해하고 다른 접근 방식들과 비교하기 위해서는 결함 모델(fault model)이 필요하다.

RAID 는 특정 종류의 결함을 감지하고 이를 복구하도록 설계되어 있기 때문에, 설계시 어떤 종류의 결함을 고려해야 하는지를 명확히 하는 것이 매우 중요하다.

첫 번째 모델은 단순한 형태로, 고장 시 멈춤(fail-stop) 결함 모델이라고 불린다. 이 모델에서는 디스크가 “정상적으로 동작”하거나 “완전히 멈춘 상태” 둘 중 하나라고 가정한다.

정상적으로 동작하는 디스크는 모든 블록을 읽고 쓸 수 있다.

반면, 멈춘 디스크는 완전히 사용할 수 없는 상태로 간주된다.

이 fail-stop 모델의 치명적인 단점은, 결함이 발생했을 때 그것을 어떻게 발견하는지에 대한 가정이다.

이 모델에서는 디스크가 고장나면 쉽게 이를 감지할 수 있다고 가정한다.

예를 들어, RAID 컨트롤러(하드웨어 또는 소프트웨어)가 디스크 고장을 즉시 인지할 수 있다고 본다.

하지만 실제 환경은 그렇게 단순하지 않다.

현재 단계에서는 디스크 섹터의 내용이 손상되는 것과 같은 “조용한(silent)오류”는 고려하지 않는다.

또한 디스크 전체가 아니라 특정 블록만 접근할 수 없는 경우(이를 잠재 섹터 오류(latent sector error)라고 부르기도 한다) 역시 고려하지 않는다.

이러한 더 복잡하고 현실적인 디스크 결함 문제들은 이후에 다룰 것이다.

### [3] RAID 의 평가 방법

이제 곧 보게 되겠지만 RAID 를 구성하는 방법에는 여러 가지가 있다.

각 구성 방법은 서로 다른 특성을 가지기 때문에, 다양한 방법들의 장점과 단점을 이해하기 위해서는 평가 기준이 필요하다

구체적으로, 세 가지 축을 중심으로 각 RAID 설계를 평가할 것이다.

#### (1) 첫 번째 축: 용량

첫 번째 축은 용량(capacity)이다.

각각 B 개의 블록을 가진 N 개의 디스크가 있다고 할 때,

RAID 를 사용하는 클라이언트가 실제로 사용할 수 있는 유효 용량은 얼마나 되는가?

- 만약 중복 저장이 없다면, 유효 용량은 당연히  $N \times B$  이다.
- 반대로, 각 블록에 대해 두 개의 복사본을 유지하는 경우(이를 미러링이라고 한다), 유효 용량은  $(N \times B) / 2$  가 된다.
- 다른 기법들(예: 패리티 기반 방식)은 이 두 경우 사이의 값을 가지게 된다.

#### (2) 두 번째 축: 신뢰성

두 번째 평가 축은 신뢰성(reliability)이다.

각 RAID 설계는 몇 개의 디스크 고장을 견딜 수 있을까?

현재의 결함 모델에서는 한 개의 디스크만 고장날 수 있다고 가정한다.

(이후 장에서는 더 복잡한 결함 상황을 다루게 된다.)

#### (3) 세 번째 축: 성능

마지막 세 번째 축은 성능이다.

성능은 디스크 배열이 처리하는 워크로드의 종류에 따라 크게 달라지기 때문에, 평가하기가 쉽지 않다.

따라서, 성능을 평가하기 전에, 먼저 고려해야할 대표적인 워크로드를 정의할 것이다.

이제 세 가지 중요한 RAID 설계를 살펴볼 것이다.

- RAID 레벨 0: 스트라이핑
- RAID 레벨 1: 미러링
- RAID 레벨 4/5: 패리티 기반 중복 저장

각 설계에 붙는 “레벨(level)”이라는 명칭은 Berkeley 대학의 Patterson, Gibbson, Katz 의 선구적인 연구에서 유래하였다.

#### [4] RAID 레벨 0: 스트라이핑

첫 번째 RAID 레벨은 사실 중복 저장을 하지 않기 때문에 엄밀히 말하면 RAID 가 아니다.

하지만 RAID 레벨 0, 또는 더 잘 알려진 이름인 스트라이핑(striping)은 성능과 용량 측면에서 훌륭한 상한 기준을 제공하기 때문에 이해할 가치가 있다.

가장 단순한 스트라이핑은 다음과 같이 동작한다.  
블록들을 여러 디스크에 걸쳐 줄을 굻듯이 분산하여 저장한다.  
(여기서는 4 개의 디스크 배열을 가정한다.)

| 디스크 0 | 디스크 1 | 디스크 2 | 디스크 3 |
|-------|-------|-------|-------|
| 0     | 1     | 2     | 3     |
| 4     | 5     | 6     | 7     |
| 8     | 9     | 10    | 11    |
| 12    | 13    | 14    | 15    |

그림을 보면 기본 개념을 이해할 수 있다.  
디스크 배열의 블록들을 라운드 로빈 방식으로 디스크들에 나누어 배치한다.  
이 방식은 연속된 큰 데이터 요청(예: 순차 읽기)이 들어왔을 때 병렬성을 최대한 활용하도록 설계되어 있다.

같은 행에 있는 블록들은 스트라이프(stripe)라고 부른다.  
따라서 블록 0, 1, 2, 3 은 같은 스트라이프에 속한다.

이 예에서는 단순히 한 블록(4KB)씩 저장한 뒤 다음 디스크로 넘어간다고 가정한다.

하지만 반드시 이렇게 할 필요는 없다.

예를 들어, 아래와 같이 배치할 수 있다.

| 디스크 0 | 디스크 1 | 디스크 2 | 디스크 3 |        |
|-------|-------|-------|-------|--------|
| 0     | 2     | 4     | 6     | 체크 크기: |
| 1     | 3     | 5     | 7     | 2 블록   |
| 8     | 10    | 12    | 14    |        |
| 9     | 11    | 13    | 15    |        |

이 경우에는 각 디스크마다 두 개의 4KB 블록을 배치한 뒤 다음 디스크로 이동한다.  
따라서 이 RAID 배열의:

- 체크 크기 8KB
- 스트라이프 크기: 4 개의 체크

#### 여담: RAID 매핑 문제

RAID 의 용량과 신뢰성 그리고 성능 특성을 공부하기 전에 매핑 문제를 다루고자 한다.

모든 RAID 배열에서 발생하는 문제로, 간단히 말하면 다음과 같다.

논리 블록에 대한 읽기와 쓰기를 할 때 RAID 는 어떻게 해당 물리 디스크와 오프셋을 알아내어 정확한 위치에 접근할 수 있는가?

간단한 RAID 레벨의 경우에는 물리 위치와 논리 블록을 연결하기 위해 복잡한 기법이 필요하지 않다.

첫 번째 스트라이핑 예제(체크 크기 = 1 블록 = 4KB)를 생각해 보자.

논리 블록 주소가 A 라고 할 때 RAID 는 다음의 두 가지 간단한 수식을 원하는 디스크와 오프셋을 계산할 수 있다.

- $\text{Disk} = A \% \text{number\_of\_disks}$
- $\text{Offset} = A / \text{number\_of\_disks}$

이 계산은 정수 계산이라는 점에 유의해야 한다.

간단한 예를 살펴보자.

첫 번째 RAID 에서 14 번 블록에 대한 요청이 있다고 하자. 디스크가 네 개라고 하면:

- 디스크 =  $14 \% 4 = 2$
- 오프셋 =  $14 / 4 = 3$

따라서 블록 14 는 디스크의 2 의 블록 3 에 해당한다.

단, 청크 크기가 바뀔 때 이 수식이 어떻게 변하는지 생각해 보라.

### 청크 크기

청크 크기는 RAID 의 성능에 큰 영향을 준다.

작은 청크 크기는 많은 파일들이 여러 디스크에 걸쳐 스트라이핑된다는 것을 의미하며, 그 결과 하나의 파일을 읽고 쓰는 데 병렬성이 증가한다.

하지만 여러 디스크에서 블록의 위치를 찾아야 하므로 위치 탐색 시간이 증가한다. 요청 처리 시간은 여러 디스크 중 가장 오래 걸린 탐색 시간에 의해 결정되기 때문이다.

반면, 큰 청크 크기의 경우 파일 내부의 병렬성은 줄어든다.

따라서 높은 처리 성능을 얻으려면 여러 요청을 동시에 실행해야 한다.

하지만 위치 탐색 시간은 감소한다.

예를 들어, 파일이 하나의 청크에 저장된다면, 이 파일을 읽는데 필요한 탐색 시간은 하나의 디스크에서의 탐색 시간과 동일하다.

최적의 청크 크기를 결정하는 것은 워크로드에 대한 깊은 이해를 필요로 하며 간단하지 않다.

이후의 논의에서는 단순화를 위해 청크 크기를 한 블록(4KB)으로 가정한다.

### RAID-0 의 분석으로 돌아가서

스트라이핑 방식의 용량, 신뢰성, 성능을 평가해 보자.

용량 측면에서는 완벽하다.

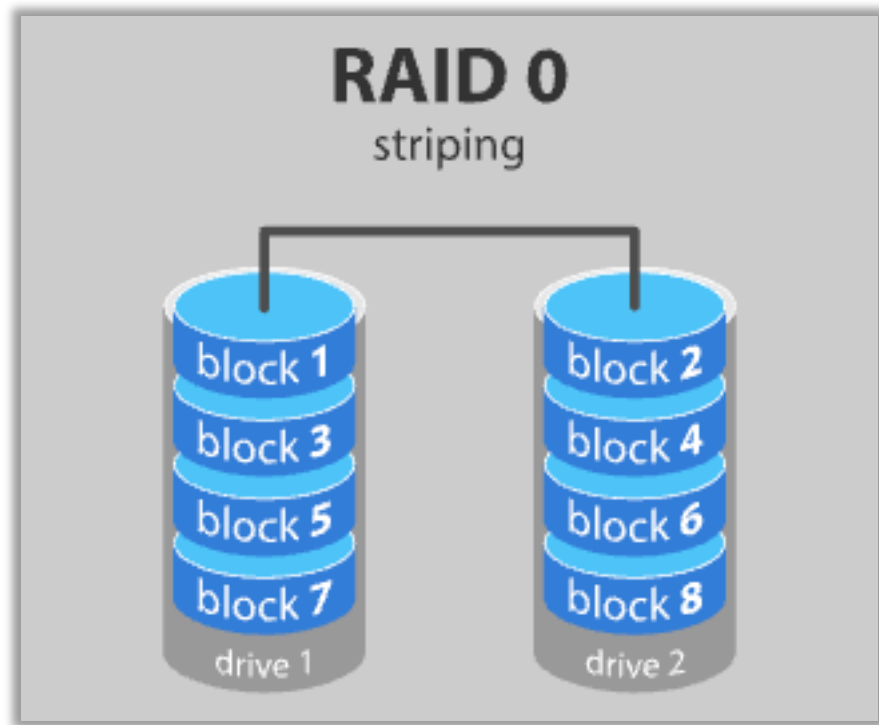
N 개의 디스크가 있을 때 유효 용량은  $N \times B$  이다.

신뢰성 측면에서는 역시 완벽하지만, 나쁜 의미에서 완벽하다.

어느 하나의 디스크라도 고장 나면 전체 데이터가 손실된다.

성능은 매우 우수하다.

모든 디스크를 병렬로 사용할 수 있기 때문이다.



### RAID 성능 평가

RAID의 성능을 분석하기 위해 두 가지 다른 성능 척도를 고려해야 한다.

하나는 단일 요청의 지연시간이다.

하나의 I/O를 처리하는 데 걸리는 RAID의 지연 시간에 대한 이해는, 한 번의 논리적 I/O 동작을 처리할 때의 병렬성 정도를 파악하는 데 도움이 된다.

두 번째 척도는 RAID의 정상 상태(steady state)에서의 처리 성능(throughput)이다. 이는 병행 요청들의 전체적인 대역폭을 의미한다.

고성능을 요구하는 환경에서 RAID가 주로 사용되기 때문에, 안정 상태에서의 대역폭은 매우 중요하며 우리의 주요 초점이 된다.

처리 성능을 좀 더 자세히 이해하기 위해, 관심을 가질만한 워크로드를 먼저 생각해보자.

이 논의를 위해 순차 워크로드와 랜덤 워크로드의 두 가지 유형이 있다고 가정한다.

순차 워크로드는 연속된 큰 청크의 형태로 RAIE 에 요청된다.

예를 들어, 1MB 의 데이터를 접근하는 요청(또는 연속적인 요청들)이 블록 B 에서 시작하여 B + 1MB 에서 끝난다면 이를 순차적이라고 본다.

여러 환경에서 순차 워크로드는 흔히 발생하며(예: 큰 파일에서 키워드를 검색하는 경우), 중요하게 여겨진다.

랜덤 워크로드는 크기가 작은 요청이며, 디스크의 여러 불특정 위치를 접근한다고 가정한다.

예를 들어, 논리 주소 10 에서 첫 번째 4KB 를 접근한 후, 다음으로 논리 주소 550,000, 그리고 20,100 등의 위치로 이동하여 접근하는 경우이다.

데이터베이스 관리 시스템의 트랜잭션과 같은 일부 중요한 워크로는 이러한 접근 패턴을 보이기 때문에, 역시 중요한 워크로드로 간주된다.

물론, 실제 워크로드는 그렇게 단순하지 않으며, 순차와 랜덤의 성질이 섞여 있기 때문에 특성도 중간적인 형태를 보인다.

단순화를 위해 이 두가지 경우만 고려한다.

순차 워크로드와 랜덤 워크로드는 디스크 성능에서 큰 차이를 만들어 낸다.

순차 접근의 경우 디스크가 가장 효율적으로 동작한다.

탐색 시간과 회전 지연이 짧고, 대부분의 시간을 데이터 전송에 사용한다.

반면, 랜덤 접근의 경우는 그 반대이다.

대부분의 시간을 탐색과 회전을 기다리는 데 사용하고, 상대적으로 적은 시간만 데이터 전송에 사용된다.

이 차이를 나타내기 위해, 디스크가 순차 워크로드에서는 S MB/s, 랜덤 워크로드에서는 R MB/s 의 속도로 전송한다고 가정한다.

일반적으로 S 는 R 보다 훨씬 크다.(즉,  $S \gg R$ )

이 차이를 이해했는지 확인하기 위해 간단한 연습 문제를 풀어보자.  
다음의 디스크 성능에 따라 S와 R을 계산해보자.

평균적으로 10MB 크기의 데이터를 순차적으로 전송하고, 평균 10KB 크기의 데이터를 랜덤 방식으로 전송한다고 가정한다.

또한 디스크의 특성은 다음과 같다.

- 평균 탐색 시간: 7 msec
- 평균 회전 시간: 3 msec
- 디스크 전송 속도: 50 MB/s

S를 계산하기 위해, 10MB를 전송하는 데 시간이 어떻게 사용되는지 보자.

먼저 탐색에 7 msec, 회전에 3 msec가 소요된다.

그 후 전송이 시작되며, 10MB를 50MB/s로 전송하는 데는 0.2 초, 즉 200 msec가 걸린다.

따라서 10MB를 전송하는 데 총 210 msec가 소요된다.

S는 다음과 같이 계산된다.

$$\begin{aligned} S &= \text{데이터 크기} / \text{접근 시간} \\ &= 10\text{MB} / 210 \text{ msec} \\ &= 47.62 \text{ MB/s} \end{aligned}$$

데이터 전송에 대부분의 시간이 사용되기 때문에, S 값은 디스크의 최대 대역폭에 근접한다.(탐색과 회전 지연의 영향은 전체 시간이 길어질수록 상대적으로 작아진다.)

R도 비슷하게 계산할 수 있다.

탐색 시간과 회전 지연은 동일하다.

전송 시간은 10KB를 50MB/s로 전송하는 경우이므로 약 0.195 msec가 걸린다.

따라서 총 시간은 약 10.195 msec이다.

R은 다음과 같이 계산된다.

$$\begin{aligned} R &= \text{데이터 크기} / \text{접근 시간} \\ &= 10\text{KB} / 10.195 \text{ msec} \\ &= 0.981 \text{ MB/s} \end{aligned}$$

보는 바와 같이 R은 1MB/s 이하이며, S/R은 약 50에 가깝다.



RAID-0 분석으로 또 다시 돌아가서  
이제 단순 스트라이핑의 성능을 평가해 보자.  
앞서 언급했듯이 전반적으로 괜찮다.

예를 들어, 지연 시간 측면에서 보면 하나의 블록에 대한 요청의 지연 시간은 하나의 디스크에 대한 지연 시간과 거의 동일하다.  
결국 RAID-0은 받은 요청을 디스크들 중 하나에 전달하는 것뿐이기 때문이다.

정상 상태에서의 대역폭 측면에서는 시스템의 최대 대역폭을 기대할 수 있다.  
따라서 처리 성능은  $N(\text{디스크 수}) \times S(\text{디스크 하나의 순차 접근 대역폭})$ 와 같다.  
또한, 많은 랜덤 I/O의 경우에도 모든 디스크를 사용할 수 있기 때문에  $N \times R$  MB/s를 얻을 수 있다.

곧 살펴보겠지만, 이 수치들은 가장 간단하게 계산될 수 있으며, 다른 RAID 레벨들과 비교할 때 상한 값으로 사용될 것이다.

## [5] RAID 레벨 1: 미러링

스트라이핑 이후의 첫 번째 RAID 레벨은 RAID 레벨 1 또는 미러링으로 알려져 있다.

미러링을 사용하는 시스템에서는 각 블록에 대해 하나 이상의 사본을 둔다.

각 사본은 당연히 서로 다른 디스크에 저장되어야 한다.

이렇게 함으로써 디스크 고장에 대응할 수 있게 된다.

일반적인 미러링 시스템에서는 RAID가 각 논리 블록에 대해 두 개의 물리적 사본을 둔다고 가정한다.

다음은 그 예이다.

| 디스크 0 | 디스크 1 | 디스크 2 | 디스크 3 |
|-------|-------|-------|-------|
| 0     | 0     | 1     | 1     |
| 2     | 2     | 3     | 3     |
| 4     | 4     | 5     | 5     |
| 6     | 6     | 7     | 7     |

이 예에서는 디스크 0과 디스크 1이 동일한 값을 가지고,

디스크 2와 디스크 3도 동일한 값을 가진다.

데이터는 이러한 미러링된 쌍들에 대해 스트라이핑되어 있다.

사실 디스크에 사본을 배치하는 방법은 매우 다양하다.

위에서 보인 방식은 일반적인 방법으로, RAID-10 또는 RAID 1+0이라고 불린다.

그 이유는 미러링(RAID-1)을 먼저 적용한 후 이를 스트라이핑(RAID-0)하기 때문이다.

또 다른 구성으로는 RAID-01 또는 RAID 0+1이 있다.

이는 배열을 먼저 RAID-0으로 스트라이핑한 후, 그 배열을 RAID-1로 미러링하는 방식이다.

여기에서는 앞에서 설명한 미러링 방식만을 가정한다.

미러링된 배열에서 블록을 읽을 때 RAID 는 원본을 읽을지, 사본을 읽을지 선택할 수 있다.

예를 들어, 논리 블록 5 를 읽으라는 요청이오면, 디스크 2 또는 디스크 3 중 하나에서 읽을 수 있다.

반면, 블록을 쓰는 경우에는 이러한 선택이 불가능하다.

RAID 는 신뢰성을 유지하기 위해 두 개의 사본을 모두 갱신해야 한다.

단, 이러한 쓰기 요청은 병렬적으로 처리될 수 있따는 점에 유의해야 한다.

예를 들어, 논리 블록 5 에 대한 쓰기 요청은 디스크 2 와 디스크 3 에서 동시에 처리될 수 있다.

### RAID-1 분석

RAID-1 을 평가해 보자.

용량 측면에서 RAID-1 은 비용이 많이 든다.

미러링 레벨이 2 라면 최대 사용 가능한 용량의 절반만 사용할 수 있다.

따라서 B 개의 블록을 가진 N 개의 디스크 배열의 경우 유효 용량은  $(N \times B) / 2$  가 된다.

신뢰성 측면에서 RAID-1 은 괜찮은 편이다.

디스크 중 어느 하나에 고장이 발생하더라도 이를 감내할 수 있다.

실제로 운이 좋다면 RAID-1 은 더 많은 고장을 감내할 수도 있다.

앞의 그림에서 디스크 0 과 디스크 2 에 동시에 고장이 발생했다고 가정해 보자.

그 경우에도 데이터를 잃지 않는다.

일반적으로 미러링 시스템(미러링 레벨 2)의 경우 하나의 디스크 고장은 확실히 감내할 수 있으며, 어떤 디스크가 고장 나느냐에 따라  $N/2$  개의 결함까지도 감내할 수 있다.

하지만 일반적으로 실제 환경에서는 이러한 운에 의존하고 싶지 않다.

따라서 대부분의 경우 하나의 고장만을 처리할 수 있다고 가정한다.

## 성능 분석

마지막으로 성능을 분석해보자.

단일 읽기 요청의 지연 시간은 단일 디스크에서 읽는 경우의 지연 시간과 동일하다. RAID-1 은 두 개의 사본 중 하나의 디스크로 요청을 전달하기 때문이다.

쓰기의 경우는 약간 다르다.

쓰기 요청이 완료되기 위해서는 두 개의 디스크에 전달된 쓰기가 모두 완료되어야 한다.

이 두 쓰기는 병렬적으로 수행되기 때문에 하나의 쓰기와 거의 동일한 시간이 걸린다.

하지만 RAID-1 의 쓰기 요청은 두 개의 물리적 쓰기 연산이 모두 완료될 때까지 기다려야 하므로, 그 지연 시간은 두 요청 중 더 오래 걸리는 탐색 시간과 회전 지연 시간에 의해 결정된다.

평균적으로 단일 디스크에 쓰는 시간보다 약간 더 길다.

## 워크로드

이제 순차 워크로드에서 정상 상태 처리 성능을 살펴보자.

디스크에 순차적으로 쓰기를 할 때, 각 논리적 쓰기는 항상 두 번의 물리적 쓰기를 필요로 한다.

예를 들어, 논리 블록 0 에 쓰기를 한다면, RAID 는 내부적으로 디스크 0 과 디스크 1 모두에 기록해야 한다.

따라서 미러링된 배열에서 순차 쓰기의 경우 처리 성능( $N/2 \times S$ ), 즉 최대 대역폭의 절반이 된다.

불행하게도 순차 읽기의 경우에도 동일한 성능을 갖는다.

순차 읽기의 경우 두 개의 사본 중 하나만 읽으면 되므로 더 높은 성능을 기대할 수도 있다.

다음의 예를 보자.

읽어야 할 블록이 0, 1, 2, 3, 4, 5, 6, 7 이라고 하자.

- 블록 0 은 디스크 0 에서 읽고
- 블록 1 은 디스크 2 에서 읽고
- 블록 2 는 디스크 1 에서 읽고
- 블록 3 은 디스크 3 에서 읽는다고 하자.

이후에도 4, 5, 6, 7 을 각각 디스크 0, 2, 1, 3 에서 읽는다.

겉으로 보면 모든 디스크를 사용하므로 전체 대역폭을 얻을 수 있을 것처럼 보인다.

하지만 그렇지 않다.

하나의 디스크(예: 디스크 0)가 받는 요청을 보면, 먼저 블록 0 을 읽고 그 다음에는 두 블록 떨어진 블록 4 를 읽는다.

즉, 각 디스크는 하나씩 건너뛴 블록에 대한 요청을 받는다.

건너뛴 블록들이 디스크 위를 지나가는 동안에는 유용한 데이터를 제공하지 못한다. 따라서 각 디스크는 자신의 최대 대역폭의 절반만 사용할 수 있다.

결과적으로 순차 읽기의 대역폭도( $N/2 \times S$ ) MB/s 가 된다.

랜덤 읽기는 미러링 RAID 에서 가장 좋은 워크로드이다.

이 경우 모든 디스크에 읽기 요청을 보낼 수 있기 때문에 최대 대역폭을 얻을 수 있다.

따라서 랜덤 읽기의 경우 RAID-1 은  $N \times R$  MB/s 의 성능을 보인다.

마지막으로 랜덤 쓰기는 예상한 대로 ( $N/2 \times R$ ) MB/s 의 성능을 가진다.

각 논리적 쓰기가 두 번의 물리적 쓰기로 변환되기 때문에,

모든 디스크를 사용하더라도 사용자는 전체 대역폭의 절반만 얻을 수 있다.

논리 블록 x 에 대해 두 개의 서로 다른 물리 디스크에 나누어 쓰기가 수행되더라도, 작은 크기의 쓰기 요청에 대한 대역폭은 스트라이핑의 경우의 절반이다.

### 여담: RAID의 일관성 있는 갱신 문제

RAID-1의 분석에 앞서, 여러 디스크로 이루어진 RAID 시스템에서 발생할 수 있는 문제인 일관성 갱신 문제(consistent-update problem)를 살펴보자.

이 문제는 하나의 논리저기 쓰기 작업을 위해 여러 디스크를 갱신해야 하는 모든 RAID에서 발생할 수 있다. 미러링된 디스크 배열을 가정하고 논의해보자.

RAID에 쓰기 요청이 들어오고, RAID가 디스크 0과 디스크 1에 기록한다고 가정하자.

RAID가 디스크 0에 쓰기 요청을 보낸 후, 디스크 1에 요청을 보내기 직전에 전력 손실(또는 시스템 중단)이 발생했다고 하자.

이와 같은 상황에서 디스크 0에 대한 쓰기 요청은 완료되었다고 가정하자.  
(하지만 디스크 1에 대한 요청은 전달되지 않았기 때문에 갱신되지 않았다.)

이와 같이 적절하지 않은 시점에 발생한 전력 손실의 결과로 두 개의 블록 내용이 서로 불일치하게 된다.

디스크 0의 사본은 새로운 값이고, 디스크 1의 사본은 이전 값이 된다.

우리가 원하는 것은 두 디스크의 상태 갱신이 원자적으로 수행되는 것이다.

즉, 두 디스크 모두 새로운 값으로 변경되거나, 아니면 아무것도 변경되지 않아야 한다.

이 문제를 해결하는 일반적인 방법은 write-ahead 로그의 일종을 사용하는 것이다.

RAID는 실제 동작을 수행하기 전에, 수행하려는 작업(예: 특정 값으로 두 디스크를 갱신하는 것)을 먼저 기록한다.

그 후 실제 갱신 작업을 수행한다.

이러한 접근 방식을 사용하면 시스템이 중단되더라도 올바른 결과를 얻을 수 있다.

RAID는 대기 중이던 모든 트랜잭션을 재실행하는 복구 절차를 수행하여,

모든 사본들(예: RAID-1의 두 사본)이 서로 일치하도록 보장할 수 있다.

마지막으로, 모든 쓰기 작업에 대해 디스크에 로그를 남기는 것은 비용이 매우 크기 때문에, 대부분의 RAID 하드웨어는 작은 크기의 비휘발성 RAM(예: 배터리로 보호되는 메모리)에 로그를 기록한다.

이렇게 하면 디스크에 직접 로그를 기록하는 비용을 들이지 않고도 일관성 있는 갱신을 수행할 수 있다.

#### [6] RAID 레벨 4: 패리티를 이용한 공간 절약

이제 패리티(parity)라고 알려진 중복성을 추가하는 또 다른 방법을 설명한다. 패리티 기반의 접근 방법은 저장 공간을 더 적게 사용하기 때문에, 미러링 기반 시스템이 지불하는 막대한 공간 낭비를 극복하려고 시도한다. 그 대신 성능이라는 비용을 지불해야 한다.

| 디스크 0 | 디스크 1 | 디스크 2 | 디스크 3 | 디스크 4 |
|-------|-------|-------|-------|-------|
| 0     | 1     | 2     | 3     | P0    |
| 4     | 5     | 6     | 7     | P1    |
| 8     | 9     | 10    | 11    | P2    |
| 12    | 13    | 14    | 15    | P3    |

여기서는 5 개의 디스크로 이루어진 RAID-4 시스템을 예로 살펴본다. 각 데이터 스트라이프마다 해당 스트라이프에 대한 중복 저장을 담고 있는 패리티 블록 하나를 추가한다.

블록 P1 은 블록 4, 5, 6, 7로부터 계산된 중복 정보를 가지고 있다.

패리티를 계산하기 위해서는 스트라이프에 속해 있는 블록 중 하나의 블록이 고장나더라도 이를 견딜 수 있도록 하는 수학적 함수가 필요하다. 간단한 함수인 XOR 이 이러한 목적에 잘 부합한다.

어떤 비트가 주어졌을 때, XOR 은 1 개의 개수가 짝수이면 0 을 반환하고, 홀수이면 1 을 반환한다.

예를 들면 다음과 같다.

|    |    |    |    |                           |
|----|----|----|----|---------------------------|
| C0 | C1 | C2 | C3 | P                         |
| 0  | 0  | 1  | 1  | $\text{XOR}(0,0,1,1) = 0$ |
| 0  | 1  | 0  | 0  | $\text{XOR}(0,1,0,0) = 1$ |

XOR: 같으면 0, 다르면 1

첫째 줄(0, 0, 1, 1)에는 C2 와 C3 에 1 이 있다.

따라서 이 값들을 XOR 하면 P 로 0 을 얻는다.

마찬가지로 두 번째 줄에는 1 이 C1 에만 있으므로, 그 줄을 XOR 하면 P 는 1 이 된다.

이것을 기억하는 간단한 방법은, 어떤 줄이든 그 줄의 1의 개수는 짝수가 되어야 한다는 것이다.

이것은 RAID의 패리티가 정확하게 동작하기 위해 반드시 유지해야 하는 불변의 조건이다.

위의 예제에서 고장난 디스크로부터 데이터를 복구하는 방법을 예상할 수 있다.

C2 열이 고장났다고 가정해 보자.

그 열에 있던 값을 알아내기 위해서는, 해당 행의 모든 값(XOR 결과인 패리티를 포함)을 읽은 후 올바른 값을 다시 계산하면 된다.

구체적으로 첫 번째 행에서 C2의 값(1)이 사라졌다고 하자.

같은 행의 다른 값들(C0의 0, C1의 0, C3의 1, 그리고 P의 0)을 읽으면 0, 0, 1, 0을 얻는다.

이 값들을 XOR 하면 해당 행의 1의 개수가 짝수가 되어야 하므로, 사라진 데이터의 값은 1이 된다.

이것이 바로 XOR 기반 패리티 기법에서 값을 복구하는 방법이다.

값을 복구할 때는 패리티 비트와 모든 비트를 사용하여 XOR 했던 것과 동일한 방식이다.

이제 한 가지 의문이 생길 수 있다.

비트들을 모두 XOR 해야 한다고 했는데, 각 디스크에 4KB(또는 그 이상)의 블록을 저장한다면 패리티를 계산하기 위해 여러 블록들을 어떻게 XOR 해야 할까?

이 역시 간단하다.

데이터 블록의 비트들에 대해 비트 단위로 XOR을 수행하면 된다.

그리고 각 비트 연산의 결과를 패리티 블록의 해당 비트 위치에 저장하면 된다.

예를 들어, 블록이 4비트 크기를 가진다고 하면(실제 4KB보다 훨씬 작지만) 다음과 같이 표현할 수 있다.

| 블록 0 | 블록 1 | 블록 2 | 블록 3 | 패리티 |
|------|------|------|------|-----|
| 00   | 10   | 11   | 10   | 11  |
| 10   | 01   | 00   | 01   | 10  |

그림에서 보는 것처럼, 패리티는 각 블록의 각 비트에 대해 수행되며, 그 결과는 패리티 블록의 해당 위치에 저장된다.



## RAID-4 분석

이제 RAID-4 를 분석해보자.

용량 측면에서 RAID-4 를 살펴보면, 패리티 정보를 저장하기 위해 각 디스크 그룹에서 하나의 디스크를 사용하므로  $(N-1) \times B$  의 저장공간을 제공한다.

신뢰성 역시 쉽게 이해할 수 있다.

RAID-4 는 오직 하나의 디스크 고장만을 감내할 수 있다.

두 개 이상의 디스크가 고장 나면, 손실된 데이터를 복원할 방법이 없다.

마지막으로 성능을 살펴보자.

먼저 정상 상태에서의 처리 성능을 살펴본다.

순차 읽기 성능의 경우, 패리티 디스크를 제외한 모든 디스크를 사용할 수 있으므로 최대 유효 대역폭은  $(N-1) \times S$  MB/s 가 된다.(쉬운 경우)

순차 쓰기 성능을 이해하려면 먼저 동작 방식을 이해해야 한다.

큰 크기의 데이터를 디스크에 쓰려고 할 때 RAID-4 는 “스트라이프 전부 쓰기(full-stripe write)”라는 간단한 최적화를 수행할 수 있다.

예를 들어, 쓰기 요청의 일부로 블록 0, 1, 2, 3 이 RAID 로 전달된 경우를 생각해보자.

이 경우, RAID 는 P0 의 새로운 값을 계산하기 위해 단순히 블록 0, 1, 2, 3 을 XOR 하면 된다.

그 후, 패리티 블록을 포함한 모든 블록을 다섯 개의 디스크에 병렬적으로 기록한다. 따라서 스트라이프 전부 쓰기는 RAID-4 에서 쓰기 작업을 수행하는 가장 효율적인 방법이다.

스트라이프 전부 쓰기를 이해했다면, RAID-4 의 순차 쓰기 성능을 계산하는 것은 쉽다.

유효 대역폭은 역시  $(N-1) \times S$  MB/s 가 된다.

패리티 디스크가 계속 사용되기도 하지만, 클라이언트는 그 디스크로부터 성능 이득을 얻을 수 없다.

이제 랜덤 읽기 성능을 살펴보자.

각 데이터 디스크에서 랜덤 읽기로 하나의 블록을 읽을 수 있지만, 패리티 디스크에서는 읽을 수 없다.

따라서 유효 성능은  $(N-1) \times R$  MB/s 가 된다.

마지막으로 랜덤 쓰기이다.

이 경우 RAID-4 에서 가장 흥미로운 상황이 발생한다.

예를 들어, 블록 1 을 갱신하려고 한다고 하자.

단순히 해당 블록만 갱신할 수도 있지만, 그렇게 하면 문제가 발생한다.

패리티 블록 P0 이 더이상 해당 스트라이프의 정확한 패리티 값을 반영하지 못하게 된다.

따라서 P0 역시 갱신되어야 한다.

이 값을 정확하고 효율적으로 갱신하는 방법에는 두 가지가 있다.

## 한 눈에 정리

### [순차 읽기]

큰 파일을 쭉 읽는 경우

- 데이터 디스크 전부 사용(패리티 제외)
- 여러 디스크에서 동시에 읽음
- 빠름

### [순차 쓰기]

- 큰 데이터를 한 번에 쓰는 경우(스트라이프 전부 쓰기)

- 동작
  1. 데이터 블록들 한 번에 받음
  2. XOR 로 패리티 계산
  3. 모든 디스크에 동시에 기록
- 빠름(읽기랑 거의 동일)

### [랜덤 읽기]

- 여기저기 작은 데이터 읽는 경우

- 동작
  - 필요한 블록이 있는 디스크만 접근
  - 패리티 디스크는 사용 안함
- 병렬 일직 가능 -> 빠름

### [랜덤 쓰기]

- 블록 하나만 수정하는 경우

- 문제
  - 블록 하나 바꾸면: 패리티도 반드시 같이 수정해야 함
- 동작
  1. 기존 데이터 블록 읽기
  2. 기존 패리티 읽기
  3. 새로운 패리티 계산
  4. 데이터 블록 쓰기
  5. 패리티 쓰기
  - 총 4 번 I/O 발생
- 느림/패리티 디스크에 모든 쓰기가 몰림
- 병목 발생

### 가산적 패리티(additive parity)

첫 번째 방법은 가산적 패리티이다.

이 방법은 다음과 같이 동작한다.

새로운 패리티 블록 값을 계산하기 위해 스트라이프 내의 다른 모든 데이터 블록(예: 블록 0, 2, 3)을 병렬적으로 읽은 뒤, 새로운 블록 1 과 함께 XOR 을 수행한다. 그 후, 새로운 데이터와 새로운 패리티를 해당 디스크에 병렬적으로 기록한다.

이 방법의 문제는 디스크 수가 증가할수록 계산량이 증가한다는 것이다. 즉, RAID 가 커질수록 패리티 계산을 위해 더 많은 읽기 연산이 필요하다.

### 감산적 패리티(subtractive parity)

이 때문에 감산적 패리티 방식이 등장하였다.

다음과 같은 비트 배열을 생각해 보자.

|    |    |    |    |                  |
|----|----|----|----|------------------|
| C0 | C1 | C2 | C3 | P                |
| 0  | 0  | 1  | 1  | XOR(0,0,1,1) = 0 |

이제 C2 를 새로운 값 C2\_new 로 갱신한다고 가정하자.

감산적 방식은 세 단계로 동작한다.

1. 기존 값 C2\_old 와 패리티 P\_old 를 읽는다.
2. 기존 값과 새로운 값을 비교한다.
3. 두 값이 같으면 패리티는 유지되고( $P_{new} = P_{old}$ ), 다르면 패리티를 발전시킨다.

즉,  $P_{old} = 1$  이면  $P_{new} = 0$ ,  
 $P_{old} = 0$  이면  $P_{new} = 1$  이 된다.

이 과정은 XOR 로 다음과 같이 표현할 수 있다.

$$P_{new} = (C_{old} \oplus C_{new}) \oplus P_{old}$$

비트가 아니라 블록 단위로 처리하므로, 블록의 모든 비트에 대해 이 연산을 수행해야 한다.

(예: 각 블록은 4096 바이트이며, 각 바이트는 8 비트를 가진다.)

대부분의 경우 새로운 블록 이전 블록과 다르기 때문에, 새로운 패리티 블록도 달라진다.

이제 언제 가산적 방식과 감산적 방식을 사용할지 생각해볼 수 있다.

가산적 방식이 감산적 방식보다 더 적은 I/O 연산을 수행하려면 몇 개의 디스크가 필요한지 고려해보자. 성능이 역전되는 디스크 수는 얼마인가?

## Small Write 문제

이제 블록 4와 13을 갱신하는 두 개의 작은 쓰기가 거의 동시에 요청된다고 가정하자.

이 데이터는 각각 디스크 0과 1에 있으며, 읽기와 쓰기는 병렬로 수행될 수 있으며 이 부분은 문제가 없다.

문제는 패리티 디스크이다.

두 요청 모두 각각의 패리티 블록(P1, P3)을 읽어야 한다.

여기서 문제가 명확해진다.

패리티 디스크는 이러한 작업 부하에서 병목이 된다.

이러한 현상을 패리티 기반 RAID의 small-write 문제라고 한다.

데이터 디스크는 병렬 접근이 가능하지만, 패리티 디스크는 병렬 처리가 불가능하다. 따라서 모든 쓰기 요청은 패리티 디스크로 인해 순차적으로 처리된다.

패리티 디스크는 하나의 논리 I/O 당

- 읽기 1회
- 쓰기 1회

총 2번의 I/O를 수행해야 한다.

따라서 RAID-4의 작은 랜덤 쓰기 성능은 패리티 디스크의 I/O 처리량을 기준으로 계산되며, 결과적으로  $(R / 2)$  MB/s가 된다.

작은 랜덤 쓰기 환경에서 RAID-4의 성능은 매우 나쁘다.

디스크를 추가하더라도 성능은 향상되지 않는다.

## I/O 지연 시간

마지막으로 RAID-4의 I/O 지연 시간을 살펴보자.

고장이 없다고 가정하면,

읽기 요청은 하나의 디스크로 전달되므로 지연 시간은 단일 디스크와 동일하다.

반면 쓰기 요청은

- 읽기 2번
- 쓰기 2번

이 필요하다.

읽기와 쓰기는 병렬로 수행될 수 있기 때문에, 전체 지연 시간은 대략 하나의 디스크 요청 지연 시간의 두 배 정도이다. (두 번의 읽기를 기다려야 하므로 최악의 경우 탐색시간이 증가할 수 있지만, 갱신 시에는 추가 탐색이 필요 없기 때문에, 평균적으로 정확히 두 배보다 작을 수 있다.)

## [7] RAID 레벨 5: 순환 패리티

Small write 문제를 해결하기 위해 (적어도 부분적으로라도), Patterson, Gibson 과 Katz 는 RAID-5 를 제안하였다.

RAID-5 는 RAID-4 와 거의 동일하게 동작하지만, 패리티 블록을 순환(rotate)시킨다는 점이 다르다.

| 디스크 0 | 디스크 1 | 디스크 2 | 디스크 3 | 디스크 4 |
|-------|-------|-------|-------|-------|
| 0     | 1     | 2     | 3     | P0    |
| 5     | 6     | 7     | P1    | 4     |
| 10    | 11    | P2    | 8     | 9     |
| 15    | P3    | 12    | 13    | 14    |
| P4    | 16    | 17    | 18    | 19    |

보는 바와 같이 각 스트라이프를 위한 패리티 블록은 RAID-4 에서의 디스크 병목 현상을 피하기 위해 전체 디스크에 걸쳐 순환 배치된다.

## [7] RAID-5 분석

RAID-5 분석의 많은 부분은 RAID-4 와 동일하다.

예를 들어, 두 레벨의 유효 용량과 결합 허용 정도는 동일하다.

따라서 순차 읽기와 쓰기 성능도 동일하다.

하나의 요청의 지연 시간(읽기이든 쓰기이든) 역시 RAID-4 와 동일하다.

모든 디스크를 다 활용할 수 있기 때문에 랜덤 읽기 성능은 약간 더 좋다.

마지막으로 랜덤 쓰기 성능은 요청들을 병렬적으로 처리할 수 있기 때문에 RAID-4 에 비해 눈에 띄게 개선된다.

블록 1 번과 블록 2 번에 대한 쓰기가 요청되었다고 해보자.

각 요청은 디스크 1 번과 디스크 4 번(블록 1 번의 패리티)에 대한 요청으로 바뀌고, 디스크 0 번과 디스크 2 번(블록 10 번과 해당 패리티)에 대한 요청으로 바뀐다.

그러므로 병렬적으로 처리할 수 있게 된다.

사실 랜덤 요청이 아주 많다면 모든 디스크들을 고르게 사용할 수 있다고 가정할 수 있다.

만약 그런 경우라면 small write 에 대한 전체 대역폭  $(N/4) * R$  MB/s 가 될 것이다.

사분의 일로 감소되는 이유는 각 RAID-5 쓰기가 여전히 총 4 번의 I/O 연산을 유발하기 때문이다.

이는 패리티 기반 RAID 를 사용할 때 어쩔 수 없이 지불해야하는 비용이다.

RAID-4 의 성능이 좋은 몇 가지 예외를 제외한다면 RAID-5 의 성능이 기본적으로 RAID-4 의 성능과 동일하기 때문에 RAID-5 는 시장에서 RAID-4 를 완전히 대체하였다.

RAID-4 가 사용되는 경우 large write 만 발생하여 small write 문제가 전혀 존재하지 않는 시스템뿐이다.

이러한 경우에는 구현이 더 간단하기 때문에 RAID-4 가 가끔 사용된다.

#### [8] RAID 비교 : 정리

|                | RAID-0      | RAID-1          | RAID-4            | RAID-5            |
|----------------|-------------|-----------------|-------------------|-------------------|
| 용량             | $N$         | $N/2$           | $N - 1$           | $N - 1$           |
| 신뢰도            | 0           | 1 (반드시)         | 1                 | 1                 |
| $N/2$ (운이 좋다면) |             |                 |                   |                   |
| 처리성능           |             |                 |                   |                   |
| 순차 읽기          | $N \cdot S$ | $(N/2) \cdot R$ | $(N - 1) \cdot S$ | $(N - 1) \cdot S$ |
| 순차 쓰기          | $N \cdot S$ | $(N/2) \cdot R$ | $(N - 1) \cdot S$ | $(N - 1) \cdot S$ |
| 랜덤 읽기          | $N \cdot R$ | $N \cdot R$     | $(N - 1) \cdot R$ | $N \cdot R$       |
| 랜덤 쓰기          | $N \cdot R$ | $(N/2) \cdot R$ | $(1/2) \cdot R$   | $(N/4) \cdot R$   |
| 지연 시간          |             |                 |                   |                   |
| 읽기             | $T$         | $T$             | $T$               | $T$               |
| 쓰기             | $T$         | $T$             | $2T$              | $2T$              |

그림은 RAID 레벨들 사이의 간단한 비교 정리를 하였다.

설명을 간단히 하기 위해 세부 사항들은 생략하였다.

예를 들어, 미러링 기반 시스템에서 쓰기 연산 시 평균 탐색 시간은 단일 디스크에 쓸 때의 시간보다 약간 더 길다.

두 개의 탐색 시간(각 디스크에 하나씩) 중 더 큰 값이 RAID-1 의 탐색 시간이 되기 때문이다.

두 개의 디스크에 대한 랜덤 쓰기 성능은 일반적으로 하나의 디스크에 대한 랜덤 쓰기 성능보다 더 낮다.

그리고 RAID-4/5 의 패리티 디스크를 갱신할 때에는 먼저 이전의 패리티를 읽어야 하는데, 이것이 최대 탐색 및 회전 지연을 유발할 수 있지만, 패리티에 두 번째로 쓸 때에는 회전 지연만 발생한다는 점도 있다.

하지만 그림의 비교는 기법 간의 핵심 차이점을 잘 보여준다.

RAID 레벨들 간의 장단점을 파악하는 데 유용하다.

지연 시간 분석에서는 단일 디스크에 대한 요청 지연 시간을 T로 표현하였다.

결론적으로, 성능만을 원하고 신뢰성을 고려하지 않는다면 스트라이핑이 당연히 최고다.

하지만 임의 I/O 성능과 신뢰성을 동시에 원한다면 미러링이 최선이다.

다만 용량 손실이 있다.

만약 용량과 신뢰성이 목적이라면 RAID-5가 가장 적합하다.

small write 성능에 대한 비용만 지불하면 된다.

마지막으로, 항상 순차 I/O 만 사용하고 용량을 극대화하고자 한다면 RAID-5가 가장 적합하다.

## [9] RAID 와 관련된 다른 흥미로운 주제들

RAID 에는 고려해야 할 몇 가지 흥미로운 개념들이 있다.

나중에 이 책에 추가될지도 모르는 내용들이다.

레벨 2 와 3, 그리고 여러 개의 디스크에 발생하는 결함에 내성이 있는 레벨 6 등 다양한 RAID 설계방법들이 존재한다.

또한 디스크가 고장났을 때 RAID 가 어떻게 대응하는지에 대한 논의도 있다.

때로는 대체용 스페어(hot spare)가 있어 고장난 디스크를 즉시 대신하기도 한다.

고장이 발생했을 때 성능에 미치는 영향, 그리고 고장난 디스크를 복구하는 과정에서 성능에 어떤 영향이 있는지도 중요한 문제이다.

또한 잠재된 섹터 오류(latent sector error)나 블록 훼손(block corruption)을 고려하는 결함 모델들도 있다.

이러한 결함들을 처리하기 위한 다양한 기법들이 존재한다(구체적인 내용은 데이터 무결성에 관한 장을 참고하라.)

마지막으로, RAID 를 소프트웨어 계층으로 구성할 수도 있다.

소프트웨어 RAID 시스템은 비용이 저렴한 장점이 있지만, 일관성 유지 갱신 문제를 포함한 여러 가지 다른 문제들을 가진다.



## [10] 요약

RAID 에 대해 논의하였다.

RAID 는 여러 개의 독립적인 디스크를 하나의 대용량이며 더 신뢰성 있는 저장 장치로 변환한다.

더 중요한 점은 이 과정을 투명하게 수행하기 때문에 상위 계층의 하드웨어와 소프트웨어는 이러한 변환 사실을 인지하지 못한다.

다양한 RAID 레벨을 사용할 수 있으며, 최종 선택은 사용자가 중요하게 생각하는 요소에 따라 결정된다.

예를 들어, 미러링 RAID 는 구조가 단순하고 신뢰성이 높으며 전반적으로 괜찮은 성능을 제공하지만, 용량 측면에서는 비효율적이다.

반면 RAID-5 는 높은 신뢰성과 우수한 용량 효율을 제공하지만, small write 에 대한 성능은 좋지 않다.

특정 워크로드에 적합한 RAID 를 선택하고, 청크 크기가 디스크 개수와 같은 매개변수를 적절히 결정하는 일은 쉽지 않으며, 과학이라기보다는 예술에 가까운 영역이다.

